

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

Báo Cáo Bài Tập Lớn

Đề tài: Nhận dạng ảnh trên dữ liệu đuôi-dài (Long-Tailed Recognition): từ huấn luyện từ đầu đến thích nghi mô hình nền tảng

Học phần: Học máy và Khai phá dữ liệu

Mã học phần: IT3190 - Mã lớp:

Giảng viên hướng dẫn: TS. Ngô Văn Linh

Nhóm sinh viên thực hiện: Tăng Trần Mạnh Hưng 20230037

Nguyễn Công Sơn 2023...

Trương Công Cảnh 2023...

Hà Nội, Ngày 16 tháng 6 năm 2026

Mục lục

Lời nói đầu	2
1 Giới thiệu đề tài	3
1.1 Đặt vấn đề	3
1.2 Tổng quan về bài toán	3
1.2.1 Phát biểu bài toán	3
1.2.2 Các thách thức chính	3
1.2.3 Mục tiêu của nhóm	4
1.3 Cách tiếp cận của nhóm chúng tôi	4
1.3.1 Câu hỏi nghiên cứu trọng tâm	4
1.3.2 Những đóng góp chính	4
2 Tiền xử lý dữ liệu	6
2.1 Đường ống xử lý dữ liệu	6
2.1.1 Thu thập dữ liệu	6
2.1.2 Mô tả tổng quan dữ liệu	7
2.1.3 Phân tích khám phá dữ liệu (EDA)	7
2.1.4 Làm sạch dữ liệu (Data Cleaning)	9
2.1.5 Kết luận	9
3 Cơ sở lý thuyết	10
3.1 Khung chung và ký hiệu	10
3.2 Track 1 — Huấn luyện từ đầu: can thiệp ở tầng nào giúp đuôi?	10
3.2.1 Baseline (Cross-Entropy)	11
3.2.2 Balanced Softmax (can thiệp ở mất mát)	11
3.2.3 Decoupling / cRT (can thiệp ở bộ phân loại)	11
3.2.4 Supervised Contrastive (SupCon, can thiệp ở biểu diễn)	11
3.2.5 CMO / Mixup / CutMix (can thiệp ở dữ liệu)	12
3.2.6 Meta-learning — Prototypical Network (trực few-shot)	12
3.3 Track 2 — Tái dùng checkpoint, không huấn luyện lại	12
3.4 Track 3 — Thích nghi mô hình nền tảng đồng bằng	13
3.4.1 CLIP zero-shot và prototype giàu bằng LLM (nguồn: ngôn ngữ)	13
3.4.2 Tip-Adapter và Tip-Adapter-F (nguồn: truy hồi)	13
3.4.3 LIFT — thích nghi nhẹ với khởi tạo ngữ nghĩa (nguồn: vision-language)	14
3.4.4 DINOv2-LIFT — mô hình nền tảng thị giác thứ hai (nguồn: thị giác)	14
3.4.5 Sinh đặc trưng bằng diffusion và feature mixup (nguồn: sinh dữ liệu / augment)	14
3.4.6 Generalized Logit Adjustment (GLA) — đo công bằng	15
3.4.7 Fusion nhận-biết-đuôi (kiểm bù trừ)	15
3.5 Pipeline đề xuất của nhóm	15
4 Các chiến lược đánh giá và độ đo áp dụng	17
4.1 Chiến lược chọn mô hình và giao thức đánh giá	17
4.1.1 Tách validation, không bao giờ dùng test	17
4.1.2 Hệ quả: validation thiếu lớp đuôi	17
4.2 Các độ đo đánh giá	17
4.3 Cấu hình thực nghiệm	18
4.3.1 Môi trường	18

4.3.2	Siêu tham số huấn luyện	18
4.4	Các mốc đối chứng (baselines)	18
5	Huấn luyện và đánh giá	19
5.1	Quy trình huấn luyện theo bốn giai đoạn	19
5.2	Kết quả định lượng trên CIFAR-100-LT	19
5.2.1	Track 1 — Leaderboard huấn luyện từ đầu	19
5.2.2	Track 2 — Tái dùng checkpoint	20
5.2.3	Track 3 — Thách nghiệm mô hình nền tảng (kết quả chính)	20
5.3	Kết quả định lượng trên CUB-200-LT (fine-grained)	21
5.4	Nghiên cứu loại trừ (Ablation Study)	22
5.4.1	Ablation A — đóng góp của pipeline so với backbone (DINOv2)	22
5.4.2	Ablation B — độ sâu fine-tune CLIP: fine-tune nặng làm hại đuôi	22
5.4.3	Augment đặc trưng: sinh dữ liệu và mixup không giúp	23
5.5	Kết quả định tính	23
5.6	Thảo luận	23
5.6.1	Các phát hiện chính (nhất quán trên cả hai dataset)	23
5.6.2	Điểm mạnh và hạn chế	24
6	Kết luận và hướng phát triển	25
6.1	Kết luận	25
6.2	Những đóng góp chính của nhóm	25
6.3	Hạn chế	25
6.4	Hướng phát triển	26
	Phụ lục	27
	Tài liệu tham khảo	28

Lời nói đầu

Mất cân bằng dữ liệu là một trong những thách thức dai dẳng và mang tính nền tảng của học máy ứng dụng. Trên thực tế, rất hiếm khi ta thu thập được một bộ dữ liệu cân bằng hoàn hảo: số ca bệnh hiếm luôn ít hơn ca bệnh phổ thông, số loài sinh vật quý hiếm luôn ít hơn loài thường gặp, và số giao dịch gian lận bao giờ cũng là thiểu số so với giao dịch hợp lệ. Vì vậy phân bố tần suất lớp thường có dạng *đuôi-dài* (long-tailed), trong đó một số ít lớp *phổ biến* (head) chiếm phần lớn dữ liệu, còn rất nhiều lớp *hiếm* (tail) chỉ có một lượng mẫu rất nhỏ. Trớ trêu là chính những lớp hiếm này lại thường là phần quan trọng nhất đối với ứng dụng.

Trong khuôn khổ học phần *Học máy và Khai phá dữ liệu* (IT3190), báo cáo trình bày kết quả nghiên cứu của nhóm về bài toán **nhận dạng ảnh trên dữ liệu đuôi-dài** (long-tailed image recognition). Thay vì chỉ áp dụng một thuật toán đơn lẻ, nhóm chọn cách tiếp cận như một *nghiên cứu so sánh có hệ thống*: triển khai và đánh giá một phổ rộng các phương pháp xử lý mất cân bằng, trải dài từ những kỹ thuật huấn luyện-từ-đầu kinh điển cho tới việc thích nghi các mô hình nền tảng (foundation model) hiện đại. Mục tiêu là trả lời câu hỏi: với các lớp đuôi chỉ có vài ảnh, đâu là cách hiệu quả nhất để cải thiện độ chính xác, và loại tri thức bên ngoài nào giúp ích nhiều nhất?

Toàn bộ mã nguồn, dữ liệu, log huấn luyện cũng như kết quả thực nghiệm được dẫn ra trong báo cáo đều là sản phẩm thực tế của nhóm, chạy trên hai bộ dữ liệu CIFAR-100-LT và CUB-200-LT. Nhóm xin chân thành cảm ơn TS. Ngô Văn Linh đã tận tình hướng dẫn và góp ý trong suốt quá trình thực hiện.

Nhóm sinh viên thực hiện

Chương 1

Giới thiệu đề tài

1.1 Đặt vấn đề

Các mô hình nhận dạng ảnh hiện đại đạt độ chính xác rất cao khi được huấn luyện trên những bộ dữ liệu *cân bằng*, nơi mỗi lớp có số lượng mẫu xấp xỉ nhau, chẳng hạn ImageNet hay CIFAR gốc. Tuy nhiên, dữ liệu thu thập trong thế giới thực gần như luôn tuân theo một phân bố tần suất **đuôi-dài** (long-tailed distribution): số lượng mẫu giảm rất nhanh khi đi từ các lớp phổ biến (head) sang các lớp hiếm (tail). Đây thực chất là phản ánh của một quy luật tự nhiên — một vài khái niệm xuất hiện thường xuyên, trong khi phần lớn khái niệm còn lại thì hiếm gặp.

Sự mất cân bằng tạo ra một cái bẫy kép cho quá trình học. Trước hết, hàm mục tiêu chuẩn (cross-entropy) tối ưu *tổng* sai số trên toàn bộ tập huấn luyện; vì các lớp head đóng góp đại đa số mẫu, mô hình bị kéo về phía dự đoán tốt cho head và gần như “bỏ rơi” các lớp tail. Tiếp đó, nếu đánh giá bằng *độ chính xác thô* (overall accuracy), chỉ số này lại bị các lớp đồng mẫu chi phối, nên có thể trông rất cao trong khi các lớp hiếm gần như không bao giờ được dự đoán đúng. Hệ quả là một mô hình “đẹp trên giấy” vẫn có thể hoàn toàn vô dụng đối với chính những lớp mà người dùng quan tâm nhất.

Bài toán càng trở nên gay gắt khi đuôi cực ngắn. Trong cấu hình mà nhóm nghiên cứu, lớp hiếm nhất chỉ có **5 ảnh** huấn luyện. Với vốn vẹn 5 ảnh, việc *học từ đầu* một bộ trích đặc trưng sâu cho lớp đó gần như là bất khả thi: mạng không có đủ tín hiệu để khái quát hoá, và mọi can thiệp ở mức hàm mất mát hay bộ lấy mẫu cũng chỉ giải quyết được một phần. Chính giới hạn này dẫn nhóm tới một câu hỏi sâu hơn: khi dữ liệu của bản thân bài toán đã quá ít, liệu ta có thể *mượn tri thức từ bên ngoài* — từ những mô hình đã được huấn luyện trên lượng dữ liệu khổng lồ — để cứu lấy đuôi hay không?

1.2 Tổng quan về bài toán

1.2.1 Phát biểu bài toán

Xét một tập huấn luyện $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^N$ gồm ảnh x_i và nhãn $y_i \in \{1, \dots, C\}$, trong đó số lượng mẫu mỗi lớp $n_c = |\{i : y_i = c\}|$ chênh lệch rất lớn. Không mất tính tổng quát, giả sử các lớp đã được sắp theo thứ tự tần suất giảm dần $n_1 \geq n_2 \geq \dots \geq n_C$. Mức độ mất cân bằng được đặc trưng bởi **hệ số mất cân bằng** (imbalance factor):

$$\text{IF} = \frac{n_{\max}}{n_{\min}} = \frac{n_1}{n_C}. \quad (1.1)$$

Mục tiêu là học một hàm phân loại f hoạt động tốt *đồng đều trên mọi lớp*. Yêu cầu này được hiện thực hoá bằng cách đánh giá trên một tập kiểm tra **cân bằng** $\mathcal{D}_{\text{test}}$, tức mỗi lớp có cùng số ảnh. Khi tập test cân bằng, độ chính xác thô trùng với *độ chính xác cân bằng* (balanced accuracy — trung bình recall trên các lớp); do đó đại lượng cần tối ưu chính là balanced accuracy và *độ chính xác trên nhóm lớp hiếm* (few-shot accuracy), chứ không phải accuracy thô.

1.2.2 Các thách thức chính

Bài toán long-tailed đặt ra một loạt thách thức đan xen. Thứ nhất là **thiên lệch của hàm mất mát**: cross-entropy chuẩn ngầm giả định prior lớp đồng đều, nên trên dữ liệu lệch, gradient bị các lớp head chi phối và đẩy biên quyết định lấn sang vùng của các lớp tail. Thứ hai là **thiên lệch của bộ phân loại**:

ngay cả khi đặc trưng đã đủ tốt, chuẩn (norm) của vector trọng số ở lớp head vẫn lớn hơn lớp tail, khiến logit của head bị thổi phồng một cách hệ thống — một dạng thiên lệch nằm ở *đầu phân loại* chứ không phải ở đặc trưng. Thứ ba là **sự khan hiếm dữ liệu cực độ ở đuôi**: với chỉ 5 ảnh mỗi lớp, phương sai ước lượng rất lớn và mô hình không thể học được biến thiên nội lớp như góc nhìn, nền hay ánh sáng chỉ từ vài ví dụ. Bên cạnh đó còn có **cạm bẫy đánh giá**: cả việc dùng accuracy thô lẫn việc chọn mô hình trực tiếp trên tập test đều có thể tạo ra kết quả lạc quan giả, nên cần một giao thức đo lường nghiêm ngặt với mô hình được chọn trên một tập validation tách riêng. Cuối cùng, độ khó còn thay đổi theo **bản chất miền dữ liệu**: trên dữ liệu *fine-grained* (phân biệt các loài chim rất giống nhau), các lớp chỉ khác biệt ở những chi tiết tinh vi, khiến việc học từ ít mẫu càng trở nên nan giải.

1.2.3 Mục tiêu của nhóm

Thay vì theo đuổi một con số state-of-the-art đơn lẻ, nhóm hướng tới việc dựng nên một **bức tranh so sánh trung thực và có chiều sâu** về các họ phương pháp xử lý long-tailed. Toàn bộ nghiên cứu được tổ chức theo một mạch tư duy rõ ràng: *đo cho đúng* → *sửa bộ phân loại* → *mượn tri thức ngoài*. Điểm thú vị của mạch này là mỗi bước tuy “rẻ” hơn (huấn luyện ít tham số hơn) nhưng lại cải thiện đuôi nhiều hơn bước trước, và đó chính là thông điệp khoa học mà báo cáo muốn chứng minh bằng số liệu thực nghiệm trên hai bộ dữ liệu.

1.3 Cách tiếp cận của nhóm chúng tôi

Toàn bộ thực nghiệm được tổ chức thành **ba lớp tiếp cận** (track), triển khai qua bốn giai đoạn (phase) liên hoàn; Hình 1.1 tóm tắt cấu trúc tổng thể này.

Track 1 — Huấn luyện từ đầu (Phase 1) sử dụng một mạng ResNet-18 huấn luyện hoàn toàn từ đầu trên dữ liệu lệch, nhằm khảo sát câu hỏi: *can thiệp ở tầng nào thì giúp được đuôi?* Các phương pháp được sắp xếp theo tầng can thiệp — mất mát (Balanced Softmax), bộ phân loại (Decoupling/cRT), biểu diễn (SupCon), dữ liệu (CMO/Mixup/CutMix), cùng một nhánh meta-learning (ProtoNet). Đây vừa là *trần* của việc học từ đầu, vừa đóng vai trò mốc đối chứng cho các track sau.

Track 2 — Tái dùng checkpoint, không huấn luyện lại (Phase 0) khai thác thêm điểm số từ các checkpoint đã có mà *không huấn luyện thêm*, thông qua ensemble (kèm TTA), tier-fusion (đầu tham số cho head, prototype cho tail), τ -normalization (chuẩn hoá norm trọng số) và CLIP fusion (trộn mô hình thị giác với CLIP zero-shot).

Track 3 — Thách nghiệm mô hình nền tảng đóng băng (Phase 2 và Phase 3) là phần trọng tâm. Thay vì học từ đầu, nhóm *đóng băng* một mô hình nền tảng (CLIP, DINOv2) và chỉ học vài tham số nhỏ trên đặc trưng đã được trích sẵn và lưu cache. Phase 2 tập trung vào việc thách nghiệm CLIP (Tip-Adapter, LIFT), còn Phase 3 nâng vấn đề lên thành một câu hỏi nghiên cứu về các *nguồn tri thức bên ngoài*.

1.3.1 Câu hỏi nghiên cứu trọng tâm

Đóng góp chính của đề tài nằm ở Phase 3 và được phát biểu thành một câu hỏi nghiên cứu rõ ràng:

Với lớp đuôi chỉ có vài ảnh, loại tri thức bên ngoài nào giúp nhiều nhất — **ngôn ngữ** (mô tả lớp do LLM sinh ra), **một mô hình nền tảng thị giác thứ hai** (DINOv2 tự giám sát), hay **sinh dữ liệu** (diffusion sinh đặc trưng đuôi) — và chúng có **bù trừ** cho nhau hay không?

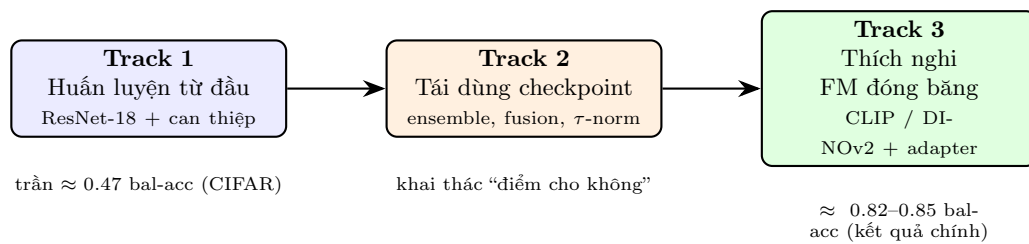
Để câu trả lời đủ độ tin cậy, nhóm khảo sát câu hỏi này trên *hai* bộ dữ liệu khác biệt cả về quy mô lớp lẫn bản chất miền: CIFAR-100-LT (100 lớp vật thể tự nhiên, IF=100) và CUB-200-LT (200 loài chim, một bài toán fine-grained, IF=10).

1.3.2 Những đóng góp chính

Bám sát mã nguồn và kết quả thực nghiệm, các đóng góp thực tế của nhóm bao gồm:

- **Một khung thực nghiệm thống nhất, đa-dataset** cho long-tailed recognition: mọi phương pháp đọc dữ liệu qua cùng một bố cục ImageFolder chuẩn, dùng chung giao thức tách validation và chọn mô hình theo balanced accuracy, nhờ đó so sánh được một cách công bằng. Khung này chạy được trên cả CIFAR-100-LT lẫn CUB-200-LT chỉ bằng việc thay nguồn dữ liệu.

- **Triển khai và đối chứng đầy đủ ba lớp phương pháp**, từ huấn luyện-từ-đầu (6 phương pháp) đến tái dùng checkpoint (4 kỹ thuật) và thích nghi mô hình nền tảng (CLIP zero-shot, Tip-Adapter, LIFT, DINOv2-LIFT, GLA, fusion), tất cả trên cùng một thước đo.
- **Một nghiên cứu so sánh có hệ thống ba nguồn tri thức ngoài** (ngôn ngữ / thị giác thứ hai / sinh dữ liệu) trên cùng một đuôi, kèm công cụ đo công bằng (GLA) và phép kiểm bù trừ (fusion nhận-biết-đuôi). Đây là phần mang đậm tính nghiên cứu nhất của đề tài.
- **Hai nghiên cứu loại trừ (ablation) làm rõ đóng góp của pipeline** so với sức mạnh của backbone: (A) tách riêng đóng góp của loss nhận-biết-đuôi và của kiến trúc adapter/cosine-head trên DINOv2; (B) khảo sát độ sâu fine-tune CLIP, qua đó chứng minh bằng số rằng *fine-tune nặng làm hại đuôi* và biện minh cho thiết kế đồng bằng.
- **Các kết luận nhất quán trên cả hai bộ dữ liệu**, trình bày kèm những *kết quả âm trung thực* (mô tả LLM không hơn zero-shot; sinh dữ liệu và mixup không mang lại cải thiện), thể hiện tinh thần khoa học khách quan.



Hình 1.1: Ba lớp tiếp cận của nhóm theo mạch *đo đúng* $\rightarrow$ *sửa bộ phân loại* $\rightarrow$ *mượn tri thức ngoài*. Càng sang phải, số tham số huấn luyện càng ít nhưng độ chính xác trên đuôi càng cao.

Chương 2

Tiền xử lý dữ liệu

2.1 Đường ống xử lý dữ liệu

Triết lý thiết kế dữ liệu của đề tài có thể tóm gọn trong một ý: mọi bộ dữ liệu dài-dài đều được quy về một bố cục chung, sao cho khi đổi dataset ta chỉ cần thay cách đọc dữ liệu chứ không phải sửa logic phương pháp. Cụ thể, mỗi dataset được tổ chức theo bố cục **ImageFolder** chuẩn, kèm hai tệp siêu dữ liệu:

```
<DATASET>-LT/  
  train/class_000/ ... class_{C-1}/    # ảnh huấn luyện (đã lấy mẫu thành long-tail)  
  test/class_000/   ... class_{C-1}/    # ảnh kiểm tra (cân bằng)  
  class_counts.json  # {"0": n0, ..., "C-1": n_{C-1}} -- số ảnh train mỗi lớp  
  class_names.json   # ["tên lớp 0", ..., "tên lớp C-1"] -- theo đúng thứ tự nhãn
```

Theo quy ước, thư mục `class_{i:03d}` tương ứng nhãn i ; `class_names[i]` là tên đọc được của lớp i (dùng cho CLIP zero-shot, sinh mô tả bằng LLM, và khởi tạo đầu phân loại của LIFT), còn `class_counts.json` mô tả hồ sơ long-tail. Hai hàm `load_class_counts` và `load_class_names` (trong `src/datasets/cifar_lt.py`) là toàn bộ chỗ “khác nhau” giữa các dataset; do phần phương pháp nhận đầu vào dưới dạng *đặc trưng + nhãn + tên lớp*, nó không cần thay đổi khi ta chuyển sang một bộ dữ liệu mới.

2.1.1 Thu thập dữ liệu

Nhóm sử dụng **hai** bộ dữ liệu nhằm kiểm tra tính tổng quát của kết luận trên hai miền và hai quy mô lớp khác nhau.

CIFAR-100-LT (IF = 100) — **bộ dữ liệu chính**. Bộ này được tạo từ CIFAR-100 gốc, gồm 100 lớp ảnh tự nhiên 32×32 RGB tổ chức thành 20 siêu lớp $\times$ 5 lớp con, với 500 ảnh train và 100 ảnh test mỗi lớp ở dạng ban đầu. Script `data/prepare_datasets.py` lấy mẫu lại tập train theo *hàm mũ* — một giao thức chuẩn trong cộng đồng long-tailed:

$$n_c = \lfloor n_{\max} \cdot \rho^{-c/(C-1)} \rfloor, \quad \rho = \text{IF} = 100, \quad n_{\max} = 500, \quad C = 100, \quad (2.1)$$

qua đó lớp head giữ 500 ảnh còn lớp tail chỉ còn 5 ảnh, tổng cộng **10 847** ảnh huấn luyện. Tập test được giữ **cân bằng** ở mức 100 ảnh/lớp (10 000 ảnh).

CUB-200-LT (fine-grained, IF = 10). Bộ này được tạo từ CUB-200-2011 (200 loài chim, kích thước ảnh biến thiên, khoảng 60 ảnh/lớp) bằng `data/prepare_cub_lt.py`. Do pool chỉ có khoảng 60 ảnh/lớp, quy trình tách **test cân bằng** 10 ảnh/lớp (2 000 ảnh) trước, rồi lấy mẫu mũ phần còn lại (khoảng 50 ảnh/lớp) theo cùng công thức (2.1) với head 50, tail 5 (IF=10), cho tổng khoảng **3 868** ảnh train. Đây là một bài toán *fine-grained*: các lớp khác nhau ở những chi tiết tinh vi như mỏ, lông hay sải cánh, nên khó hơn nhiều so với phân loại vật thể thông thường. Vì pool tối đa chỉ khoảng 50 ảnh/lớp nên CUB chỉ đạt $\text{IF} \leq 50$; nhóm trình bày nó như một miền fine-grained và *không* so trực tiếp hệ số IF với CIFAR.

2.1.2 Mô tả tổng quan dữ liệu

Bảng 2.1 tổng hợp các thống kê then chốt của hai bộ dữ liệu sau khi lấy mẫu long-tail (số liệu CIFAR lấy trực tiếp từ `outputs/cifar/summary.csv`).

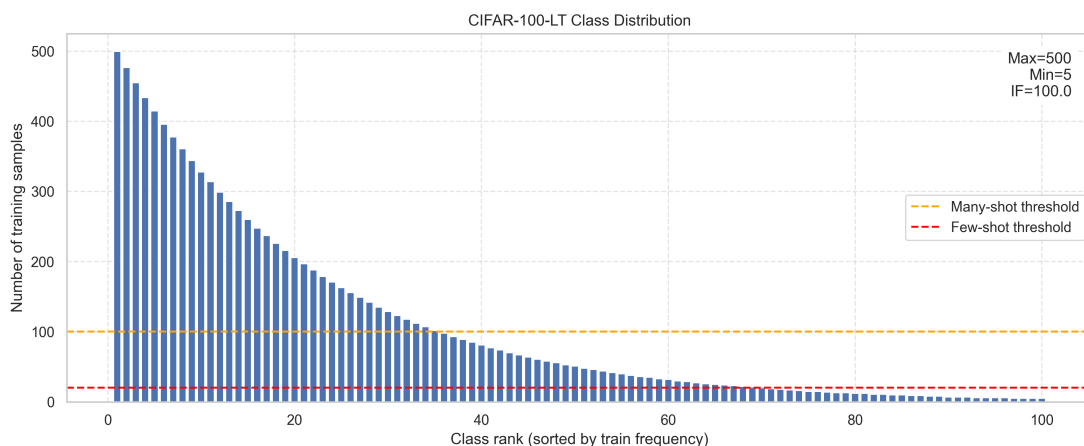
Bảng 2.1: Thống kê tổng quan hai bộ dữ liệu sau khi lấy mẫu long-tail.

Thuộc tính	CIFAR-100-LT	CUB-200-LT
Số lớp C	100	200
Tổng ảnh train	10 847	~3 868
Tổng ảnh test (cân bằng)	10 000	2 000
Số ảnh lớp lớn nhất ($n_{\max}$)	500	50
Số ảnh lớp nhỏ nhất ($n_{\min}$)	5	5
Trung bình ảnh/lớp	108.47	~19.3
Trung vị ảnh/lớp	49.5	—
Hệ số mất cân bằng (IF)	100	10
Kích thước ảnh	32×32	biến thiên ($\rightarrow$ resize)
Bản chất miền	vật thể tự nhiên	fine-grained (loài chim)
Nhóm many / medium / few (số lớp)	35 / 35 / 30	78 / 66 / 56
Ngưỡng nhóm-shot	>100 / 20–100 / <20	>20 / 10–20 / <10

Để có thể báo cáo chi tiết theo mức độ hiếm, các lớp được chia thành ba nhóm *many* (nhiều mẫu), *medium* (trung bình) và *few* (hiếm), với ngưỡng chọn sao cho ba nhóm cân đối về số lớp. Với CIFAR (head 500), ngưỡng là 100 và 20, cho 35/35/30 lớp. Với CUB (head chỉ 50), nếu giữ nguyên ngưỡng 100/20 thì nhóm *many* sẽ rỗng; do đó nhóm dùng ngưỡng riêng 20/10, cho 78/66/56 lớp. Đây là một quyết định thiết kế quan trọng đã được phản ánh trong mã nguồn — mọi lời gọi `split_shot_groups` đều phải truyền ngưỡng phù hợp với từng dataset.

2.1.3 Phân tích khám phá dữ liệu (EDA)

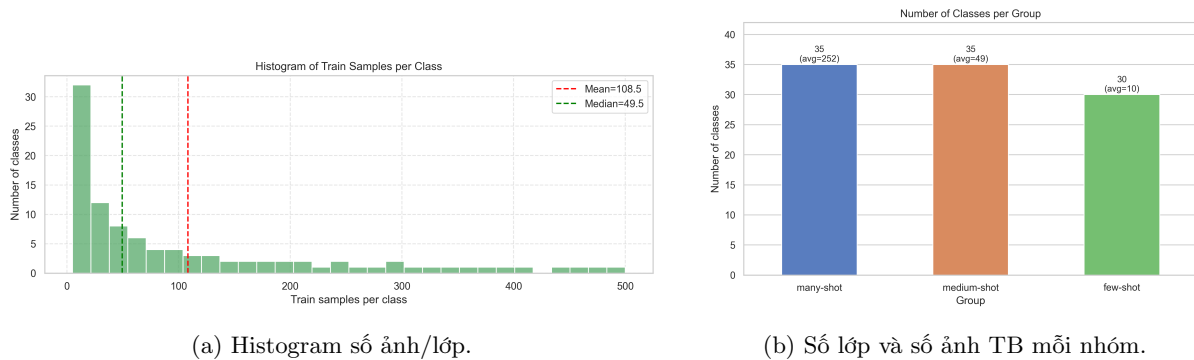
Hình dạng đuôi-dài. Hình 2.1 vẽ số ảnh huấn luyện theo thứ hạng lớp (sắp giảm dần) trên CIFAR-100-LT. Đường cong tụt rất nhanh đúng theo dạng mũ của công thức (2.1): chỉ một số ít lớp đầu sở hữu vài trăm ảnh, còn phần lớn các lớp nằm sát đáy. Hai đường ngưỡng many-shot (100) và few-shot (20) chia phổ thành ba vùng. Phân bố này chính là nguyên nhân gốc rễ của mọi khó khăn về sau, bởi nếu tối ưu tổng loss thì mô hình “được lời” nhiều hơn khi tập trung vào phần đầu của đường cong.



Hình 2.1: Phân bố số ảnh huấn luyện theo thứ hạng lớp trên CIFAR-100-LT (IF=100, $n_{\max} = 500$, $n_{\min} = 5$). Đường đứt cam = ngưỡng many-shot, đường đứt đỏ = ngưỡng few-shot.

Độ lệch của phân bố. Histogram số ảnh/lớp ở Hình 2.2 (trái) cho thấy phần lớn các lớp dồn về vùng ít mẫu (mode nằm ở khoảng 5–20 ảnh), trong khi đuôi phải kéo dài tới 500. Việc trung bình (108.5) lớn

hơn nhiều so với trung vị (49.5) là một dấu hiệu kinh điển của phân bố lệch phải mạnh. Phần bên phải của hình minh họa rõ hơn sự chênh lệch: số ảnh trung bình mỗi nhóm cách nhau hơn 25 lần, với many ≈ 252 , medium ≈ 49 và few ≈ 10 ảnh/lớp.

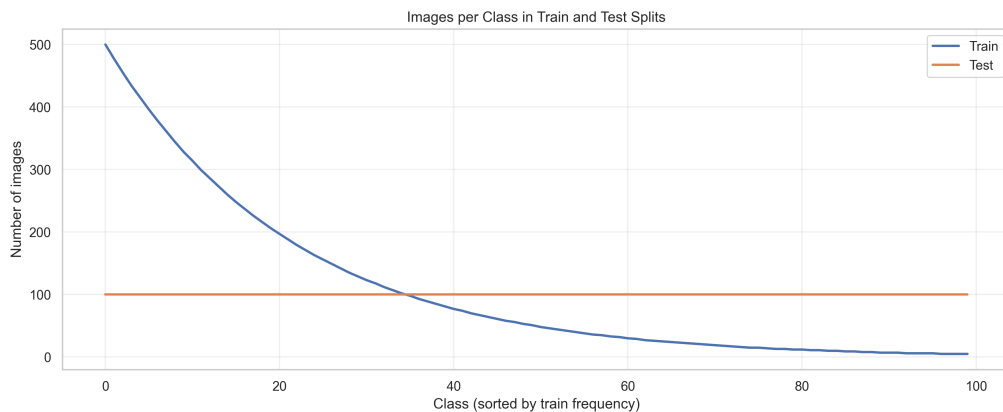


(a) Histogram số ảnh/lớp.

(b) Số lớp và số ảnh TB mỗi nhóm.

Hình 2.2: Mức độ lệch của CIFAR-100-LT. Trung bình (108.5) $\gg$ trung vị (49.5); số ảnh trung bình của nhóm few (≈ 10) nhỏ hơn nhóm many (≈ 252) hơn 25 lần.

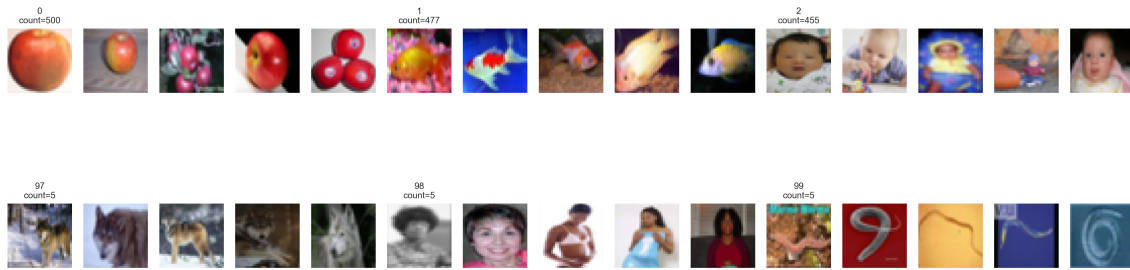
Train lệch nhưng test cân bằng. Hình 2.3 chồng số ảnh/lớp của tập train (đường giảm mũ) lên tập test (đường ngang ở mức 100). Đây là điểm mấu chốt của giao thức đánh giá: ta huấn luyện trên phân bố lệch nhưng đo trên phân bố đều, mô phỏng đúng tình huống thực tế khi dữ liệu thu được hiếm khi cân bằng nhưng mọi lớp lại quan trọng như nhau lúc triển khai. Cũng chính vì test cân bằng nên ta có $\text{accuracy} = \text{balanced_accuracy}$.



Hình 2.3: Số ảnh mỗi lớp ở tập train (giảm mũ) so với test (cân bằng 100/lớp) trên CIFAR-100-LT.

Quan sát định tính head và tail. Hình 2.4 hiển thị một số ảnh đại diện của các lớp head (hàng trên) và tail (hàng dưới). Quan sát cho thấy bản thân ảnh tail không hề “khó hơn” ảnh head về mặt chất lượng; thứ chúng thiếu chỉ là *số lượng*. Điều này củng cố giả thuyết của nhóm: nút thắt không nằm ở việc nhận diện đối tượng mà ở chỗ *ước lượng đủ tốt từ quá ít mẫu* — một khó khăn mà tri thức bên ngoài hoàn toàn có thể bù đắp.

Top Row = Head Classes, Bottom Row = Tail Classes



Hình 2.4: Ảnh mẫu của các lớp head (hàng trên, ~500 ảnh) và tail (hàng dưới, ~5 ảnh). Ảnh tail không kém chất lượng — chúng chỉ *quá ít*.

2.1.4 Làm sạch dữ liệu (Data Cleaning)

Do dữ liệu xuất phát từ hai benchmark đã được kiểm định kỹ (CIFAR-100 và CUB-200-2011), gần như không có nhãn sai hay ảnh hỏng cần loại bỏ. Vì vậy công việc “làm sạch” ở đây chủ yếu xoay quanh việc *chuẩn hoá tiền xử lý* để mọi phương pháp nhận đầu vào hợp lệ và so sánh được công bằng.

Trước hết, một tỉ lệ $\text{VAL_FRACTION} = 0.1$ được tách *phân tầng theo lớp* từ tập train để chọn checkpoint, trong khi tập test không bao giờ tham gia vào quá trình chọn mô hình. Một ràng buộc quan trọng là mỗi lớp phải giữ ít nhất một ảnh train, nên các lớp tail (vốn chỉ có 5 ảnh) được giữ nguyên cả 5; hệ quả là tập validation thường *không chứa lớp tail* — một đặc điểm sẽ ảnh hưởng trực tiếp tới cách tinh chỉnh siêu tham số ở Chương 4. Về phép biến đổi ảnh, với ảnh 32×32 (CIFAR-native) nhóm dùng random crop (padding 4), random horizontal flip và chuẩn hoá theo mean/std của CIFAR khi huấn luyện, còn khi test chỉ chuẩn hoá; với ảnh kích thước biến thiên (CUB) hoặc khi dùng mô hình nền tảng, `build_transforms` ép resize về vuông ($s \times s$) để các ảnh gộp batch được, sau đó áp tiền xử lý riêng của từng backbone (CLIP resize 224; DINOv2 resize/center-crop 224 kèm chuẩn hoá ImageNet). Để tránh “cứng hoá” các giả định riêng của CIFAR (mỗi giả định như vậy từng gây ra một lỗi thực sự đã được sửa), `NUM_CLASSES` được suy ra từ `class_counts` và tên lớp lấy từ `load_class_names`. Cuối cùng, ở Track 3, đặc trưng của mô hình nền tảng được trích *một lần* cho mỗi split rồi lưu cache trên CPU, nhờ đó việc huấn luyện adapter về sau chỉ tốn vài giây mỗi epoch.

2.1.5 Kết luận

Phân tích dữ liệu khẳng định ba điều định hình toàn bộ thiết kế phương pháp về sau. Thứ nhất, phân bố train cực kỳ lệch (IF=100, đuôi 5 ảnh) trong khi test cân bằng, nên **balanced accuracy** và **few-shot accuracy** mới là thước đo đúng, còn accuracy thô sẽ đánh lừa. Thứ hai, đuôi quá ngắn khiến việc học-từ-đầu cho các lớp hiếm gần như vô vọng, gợi ý mạnh rằng cần *mượn tri thức ngoài*. Thứ ba, việc bổ sung CUB-200-LT (fine-grained) cho phép kiểm chứng kết luận trên một miền khó hơn và một quy mô lớp gấp đôi. Những quan sát này là cơ sở trực tiếp cho các lựa chọn phương pháp được trình bày ở Chương 3.

Chương 3

Cơ sở lý thuyết

Chương này trình bày nền tảng lý thuyết của tất cả các phương pháp được nhóm triển khai, theo đúng ba lớp tiếp cận đã giới thiệu. Với mỗi phương pháp, nhóm cố gắng tái hiện trọn vẹn mạch suy nghĩ: *động lực* dẫn tới nó, *trực giác* đằng sau, *công thức toán học* đúng như được cài đặt, cùng những *ưu và nhược điểm* so với các lựa chọn khác. Phần cuối chương tổng hợp tất cả thành *pipeline đề xuất* của nhóm.

3.1 Khung chung và ký hiệu

Bộ trích đặc trưng (backbone). Ở Track 1, nhóm dùng **ResNet-18** với một sửa đổi quan trọng dành cho ảnh nhỏ: thay “stem” gốc của ImageNet (conv 7×7 stride 2 + maxpool) bằng một conv 3×3 stride 1 và bỏ hẳn maxpool. Lý do là stem gốc thu nhỏ ảnh 32×32 xuống còn 4×4 trước khi đặc trưng kịp hình thành, làm hỏng tín hiệu trên CIFAR. Đây là giao thức chuẩn của CIFAR-LT (**pretrained=False**, huấn luyện từ đầu), với đầu ra là vector đặc trưng 512 chiều sau global average pooling. Cấu hình thay thế **pretrained=True** giữ nguyên stem ImageNet và đòi hỏi ảnh 224, nên chỉ được dùng cho bảng phụ.

Chọn mô hình theo balanced accuracy. Một quyết định nền tảng khác là checkpoint tốt nhất luôn được chọn theo **balanced accuracy trên tập validation** thay vì accuracy thô. Vì validation vẫn giữ hình dạng long-tail, accuracy thô sẽ thiên về head và vô tình *trừng phạt* chính các phương pháp nhận-biết-đuôi. Quy ước này được áp dụng đồng nhất cho mọi phương pháp huấn luyện nhằm bảo đảm so sánh công bằng.

3.2 Track 1 — Huấn luyện từ đầu: can thiệp ở tầng nào giúp đuôi?

Track 1 khảo sát một câu hỏi có cấu trúc: nếu cố định backbone và chỉ can thiệp ở những *tầng* khác nhau — mất mát, bộ phân loại, biểu diễn hay dữ liệu — thì đuôi cải thiện ra sao? Bảng 3.1 liệt kê các phương pháp theo tầng can thiệp.

Bảng 3.1: Các phương pháp Track 1 phân theo tầng can thiệp.

Phương pháp	Tầng can thiệp	Ý tưởng cốt lõi
baseline	—	ResNet-18 + cross-entropy (mốc tham chiếu)
balanced_softmax	mất mát	cộng log class-prior vào logit
decoupling (cRT)	bộ phân loại	học đặc trưng rồi huấn luyện lại head cân bằng
supcon	biểu diễn	học tương phản có giám sát + cRT
cmo	dữ liệu	CutMix thiên-đuôi + Balanced Softmax
meta	few-shot	episodic Prototypical Network

3.2.1 Baseline (Cross-Entropy)

Mô hình tham chiếu là ResNet-18 gắn một đầu tuyến tính, huấn luyện bằng cross-entropy chuẩn $\mathcal{L}_{\text{CE}} = -\frac{1}{B} \sum_i \log p_{y_i}(x_i)$ với $p = \text{softmax}(z)$. Cross-entropy ngầm giả định prior lớp đồng đều; trên dữ liệu lệch, chuẩn trọng số $\|w_c\|$ tăng theo tần suất lớp, làm logit head bị thổi phồng đến mức lớp tail gần như không bao giờ thắng được phép argmax. Đây chính là hiện tượng mà các phương pháp tiếp theo tìm cách khắc phục.

3.2.2 Balanced Softmax (can thiệp ở mất mát)

Động lực. Nếu gốc rễ của vấn đề là mô hình ngầm giả định prior đồng đều, thì cách dễ nhất để sửa là *nói cho nó biết prior thật* và bù trừ ngay trong logit. Balanced Softmax (Ren và cộng sự, NeurIPS 2020) làm đúng điều đó.

Công thức. Gọi $\pi_c = n_c / \sum_j n_j$ là prior lớp ước lượng từ tần suất train. Logit được dịch bằng log-prior trước khi đưa vào cross-entropy:

$$\mathcal{L}_{\text{BS}} = \text{CE}(z + \log \pi, y), \quad \log \pi_c = \log \frac{n_c}{\sum_j n_j}. \quad (3.1)$$

Trong cài đặt, $\log \pi$ được lưu như một buffer và chỉ cộng vào lúc huấn luyện; khi test ta dùng logit thô (giả định test cân bằng). Trực giác là lớp head có $\log \pi_c$ lớn (ít âm) nên bị “phạt” nhiều hơn, còn lớp tail được “trợ giá”, nhờ vậy gradient được cân bằng lại.

Ưu/nhược. Phương pháp này không thêm tham số, không đổi kiến trúc, chỉ thay một dòng tiêu chí — vừa rất dễ vừa hiệu quả. Hạn chế của nó là không tác động tới phần *biểu diễn*: nếu đặc trưng của lớp tail vốn đã nghèo nàn vì quá ít dữ liệu thì việc điều chỉnh logit cũng chỉ giúp được tới một giới hạn nhất định.

3.2.3 Decoupling / cRT (can thiệp ở bộ phân loại)

Động lực. Kang và cộng sự (ICLR 2020) chỉ ra rằng trên long-tail, *đặc trưng học được thực ra đã đủ tốt*, cái lệch nằm ở *bộ phân loại*. Từ nhận định đó, họ đề xuất tách (decouple) thành hai pha: học biểu diễn trước, rồi huấn luyện lại riêng đầu phân loại.

Thuật toán. Pha thứ nhất huấn luyện baseline bình thường trên dữ liệu lệch để thu được một encoder tốt. Pha thứ hai *đóng băng encoder*, thay đầu tuyến tính bằng một đầu mới khởi tạo lại, rồi huấn luyện lại trong ít epoch (*class-balanced re-training*, cRT) với **bộ lấy mẫu cân bằng lớp** có trọng số tỉ lệ $1/n_c$. Một chi tiết cài đặt then chốt là khi cRT, encoder được đặt ở chế độ *eval* để *đóng băng thống kê BatchNorm* (cờ `eval_encoder`); nếu không, thống kê BN sẽ trôi theo phân bố cân bằng và làm hỏng đặc trưng đã đóng băng.

Ưu/nhược. Cách làm này tách bạch rõ nguồn thiên lệch và rất dễ ở pha hai. Tuy nhiên chất lượng cuối cùng vẫn bị khóa bởi đặc trưng học ở pha một: nếu đuôi quá nghèo, việc cân bằng lại head cũng không thể tạo ra đặc trưng mới.

3.2.4 Supervised Contrastive (SupCon, can thiệp ở biểu diễn)

Động lực. Nếu một phần nút thắt nằm ở *biểu diễn*, ta nên học một không gian đặc trưng nơi các mẫu cùng lớp tụm lại và khác lớp tách xa, độc lập với đầu phân loại. SupCon (Khosla và cộng sự, 2020) là một hàm mất mát tương phản có giám sát phục vụ đúng mục đích này.

Công thức. Với mỗi mẫu neo i trong batch (embedding z đã chuẩn hoá L_2 , nhiệt độ τ), gọi $P(i)$ là tập mẫu cùng lớp:

$$\mathcal{L}_{\text{SupCon}} = \sum_i \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i^\top z_p / \tau)}{\sum_{a \neq i} \exp(z_i^\top z_a / \tau)}. \quad (3.2)$$

Cài đặt sử dụng biến thể *two-crop* (mỗi ảnh hai phép tăng cường), một đầu chiếu (projection head) dạng MLP $512 \rightarrow 512 \rightarrow 128$ và $\tau = 0.07$. SupCon *chỉ* pretrain trên tập train để tránh rò rỉ validation; sau pretrain, encoder được đóng băng và áp cRT để tạo đầu phân loại cân bằng.

Ưu/nhược. SupCon có tiềm năng tạo ra biểu diễn mạnh hơn, nhưng thực nghiệm cho thấy một hạn chế rõ: trên đuôi cực ngắn, số cặp dương của lớp tail quá ít nên SupCon khó hình thành cụm tốt cho tail (kết quả thực nghiệm sẽ xác nhận điều này).

3.2.5 CMO / Mixup / CutMix (can thiệp ở dữ liệu)

Động lực. Nếu dữ liệu tail quá thiếu, một hướng tự nhiên là *tổng hợp thêm* ví dụ tail từ những gì đang có. Mixup và CutMix là hai phép tăng cường trộn ảnh, còn CMO (Context-rich Minority Oversampling, Park và cộng sự, CVPR 2022) là biến thể thiên-đuôi.

Công thức. Với hệ số trộn $\lambda \sim \text{Beta}(\alpha, \alpha)$, ba biến thể được định nghĩa như sau:

- *Mixup*: $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$;
- *CutMix*: dán một mảnh chữ nhật của x_j lên x_i , với λ bằng tỉ lệ diện tích giữ lại;
- *CMO*: tương tự CutMix nhưng *ảnh nền* x_i lấy từ luồng *instance-balanced* còn *mảnh* dán x_j lấy từ luồng *class-balanced* (giàu lớp hiếm); nhờ vậy lớp hiếm được dán lên nhiều bối cảnh head đa dạng, tạo ra các ví dụ tail mới giàu ngữ cảnh.

Cả ba đều dùng mất mát hai-nhân-mềm $\lambda \text{CE}(\tilde{x}, y_i) + (1 - \lambda)\text{CE}(\tilde{x}, y_j)$. Trong project, *cmo* là một phương pháp *gộp*: CMO ($\alpha = 1$, xác suất trộn 0.5) kết hợp với Balanced Softmax làm tiêu chí, nên nó can thiệp đồng thời ở cả tầng *dữ liệu* và tầng *mất mát*.

Ưu/nhược. CMO làm giàu đuôi một cách trực tiếp mà không cần mạng phụ, và thực nghiệm cho thấy đây là phương pháp from-scratch mạnh nhất. Dù vậy, các ảnh trộn vẫn bị giới hạn bởi 5 ảnh gốc của lớp tail nên không thể tạo ra ngữ nghĩa thực sự mới.

3.2.6 Meta-learning — Prototypical Network (trực few-shot)

Động lực. Nếu vấn đề cốt lõi là “ít mẫu”, vậy hãy *huấn luyện mô hình để giải học từ ít mẫu*. ProtoNet huấn luyện theo từng *episode* N -way K -shot, mô phỏng đúng kịch bản few-shot.

Công thức. Mỗi episode lấy N lớp, mỗi lớp K ảnh support cùng một số ảnh query. Prototype của lớp c là trung bình đặc trưng support, $\mu_c = \frac{1}{K} \sum_{x \in S_c} \phi(x)$, và query được phân loại theo *khoảng cách Euclid bình phương âm* $\text{logit}_c(x) = -\|\phi(x) - \mu_c\|_2^2$ rồi đưa qua cross-entropy. Cấu hình dùng là 5-way 5-shot 15-query với Adam lr = 10^{-3} . Khi đánh giá đủ 100-way, prototype được tính trên toàn bộ train rồi gán theo nearest-prototype.

Ưu/nhược. Vì phân loại theo khoảng cách nên ProtoNet *bất biến* với chuẩn trọng số, qua đó tránh được kiểu thiên lệch head của đầu tuyến tính. Mặt khác, ProtoNet vốn thiết kế cho few-way nên hoạt động yếu ở 100-way (đúng như kỳ vọng); do đó nhóm báo cáo nó như một nhánh *bonus* trên trục few-shot chứ không kỳ vọng nó dẫn đầu bảng 100-way.

3.3 Track 2 — Tái dùng checkpoint, không huấn luyện lại

Track 2 trả lời câu hỏi: liệu có thể vắt thêm điểm số mà không cần huấn luyện lại? Tất cả các kỹ thuật ở đây chỉ chạy ở pha inference trên những checkpoint sẵn có từ Track 1.

Kỹ thuật đầu tiên là **ensemble + TTA**, lấy trung bình xác suất của nhiều mô hình $\bar{p} = \frac{1}{M} \sum_m p^{(m)}$ rồi argmax. Test-time augmentation (TTA) lật ngang ảnh và trung bình thêm; vì lỗi của các mô hình không tương quan hoàn toàn nên việc trung bình giúp giảm phương sai. Cách này rẻ nhưng chủ yếu cải thiện head.

Tier-fusion trộn xác suất *tham số* (đầu tuyến tính, mạnh ở head) với xác suất *prototype* (nearest-class-mean, mạnh ở tail) theo trọng số *phụ thuộc nhóm-shot*: $p_{\text{blend}}[c] = (1 - w_{g(c)}) p_{\text{param}}[c] + w_{g(c)} p_{\text{proto}}[c]$,

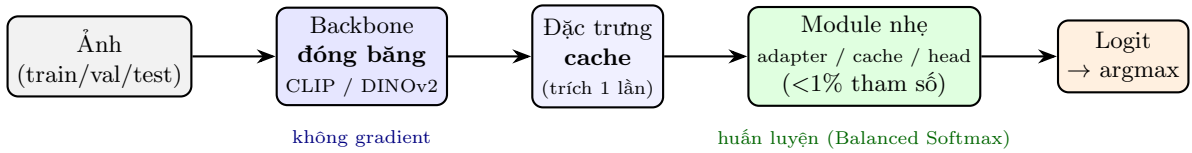
với trọng số mặc định $w_{\text{many}}=0$, $w_{\text{medium}}=0.3$, $w_{\text{few}}=0.8$. Đây là sự vận dụng bài học của meta-learning — tin prototype hơn ở đuôi — và vì trọng số được áp theo *cột-lớp* nên ta không cần biết nhãn test.

τ -normalization trực tiếp gỡ thiên lệch chuẩn trọng số (Kang và cộng sự, 2020) bằng cách chuẩn hoá lại trọng số lớp $\hat{w}_c = w_c / \|w_c\|^\tau$, với τ chọn trên validation. Giá trị $\tau=0$ giữ nguyên trọng số, còn $\tau=1$ là chuẩn hoá hoàn toàn. Vì norm của head lớn hơn, việc chia cho norm mũ τ sẽ “xếp” logit head và trả lại công bằng cho tail mà không cần huấn luyện lại bất cứ thứ gì.

Cuối cùng, **CLIP fusion (Vision-Language)** đóng vai trò bước chuyển tiếp sang Track 3. Mô hình thị giác nhỏ giỏi head nhưng kém tail, trong khi CLIP zero-shot nhận diện qua *tên lớp* và không hề thấy dữ liệu lệch của ta nên mạnh đều ở mọi lớp. Ta trộn lỗi xác suất của hai chuyên gia, $p = \alpha p_{\text{CLIP}} + (1 - \alpha) p_{\text{vision}}$, với α là *một số chung* chọn trên validation theo balanced accuracy. Ở đây không có chuyện “đoán mẫu nào hiếm”; ta chỉ trộn cho mọi lớp rồi argmax.

3.4 Track 3 — Thích nghi mô hình nền tảng đóng băng

Đây là phần trọng tâm của đề tài, xuất phát từ một luận điểm: 5 ảnh là không đủ để *học từ đầu*, nhưng đủ để *thích nghi nhẹ* một mô hình nền tảng vốn đã hiểu thế giới. Mọi phương pháp trong track này đều **đóng băng backbone** và chỉ học rất ít tham số trên *đặc trưng đã trích sẵn và cache*, nên cực nhẹ và phù hợp với môi trường Kaggle. Hình 3.1 mô tả pipeline chung.



Hình 3.1: Pipeline chung của Track 3: đóng băng backbone, trích và cache đặc trưng một lần, rồi chỉ huấn luyện một module nhẹ (<1% tham số) bằng loss nhận-biết-đuôi.

3.4.1 CLIP zero-shot và prototype giàu bằng LLM (nguồn: ngôn ngữ)

CLIP zero-shot. CLIP gán ảnh và văn bản vào cùng một không gian. Với mỗi lớp, ta mã hoá câu mẫu “a photo of a {tên lớp}” thành vector văn bản chuẩn hoá t_c ; logit zero-shot là độ tương đồng cosine có nhiệt độ $z_c(x) = s \cdot \hat{\phi}(x)^\top t_c$ với $s = \exp(\text{logit_scale})$. Vì không huấn luyện gì và không thấy dữ liệu lệch, CLIP không mang thiên lệch head/tail. Nhóm sử dụng **ViT-B/32** (cấu hình nhẹ nhất).

Prototype giàu bằng LLM (CuPL). Câu “a photo of a {” khá mỏng về thông tin. Theo tinh thần CuPL (Pratt và cộng sự, ICCV 2023), nhóm để một LLM nhỏ (**Qwen2.5-3B-Instruct**) sinh khoảng 8 mô tả thị giác cho mỗi lớp (về hình dáng, màu sắc, kết cấu...), mã hoá tất cả bằng CLIP rồi *trung bình* lại thành một prototype văn bản giàu hơn t_c . Các mô tả này được sinh một lần và cache, dùng cho cả zero-shot lẫn khởi tạo đầu LIFT.

3.4.2 Tip-Adapter và Tip-Adapter-F (nguồn: truy hồi)

Trực giác. Ý tưởng ở đây là phân loại bằng *truy hồi*: ảnh test giống ảnh train nào trong “bộ nhớ” thì bầu theo lớp đó (k-NN mềm), rồi cộng với điểm zero-shot của CLIP. Đây chính là Tip-Adapter (Zhang và cộng sự, ECCV 2022), một phương pháp *không cần huấn luyện*.

Công thức. Cache gồm *khoá* $\mathbf{F}$ (đặc trưng train) và *giá trị* $\mathbf{L}$ (nhãn one-hot). Với truy vấn $\hat{\phi}(x)$:

$$\text{logit}_{\text{Tip}}(x) = \underbrace{s \hat{\phi}(x)^\top \mathbf{T}^\top}_{\text{zero-shot}} + \underbrace{\alpha \cdot \exp(-\beta(1 - \hat{\phi}(x)\mathbf{F}^\top)) \mathbf{L}}_{\text{cache (k-NN mềm)}}, \quad (3.3)$$

trong đó β điều khiển độ sắc của trọng số lân cận còn α là mức tin cache; cả hai được *grid-search* trên *validation* theo balanced accuracy ($\alpha \in \{1, 2, 5, \dots, 100\}$, $\beta \in \{1, 2, 3, 5, 7\}$). Một điểm tinh tế cho long-tail là cache mặc định sẽ lệch về head (head có nhiều khoá hơn), nên nhóm dùng **cache cân bằng**: chia mỗi cột-lớp cho số mẫu của lớp đó, biến tổng thành *trung bình* để lớp hiếm có tiếng nói ngang với lớp phổ biến.

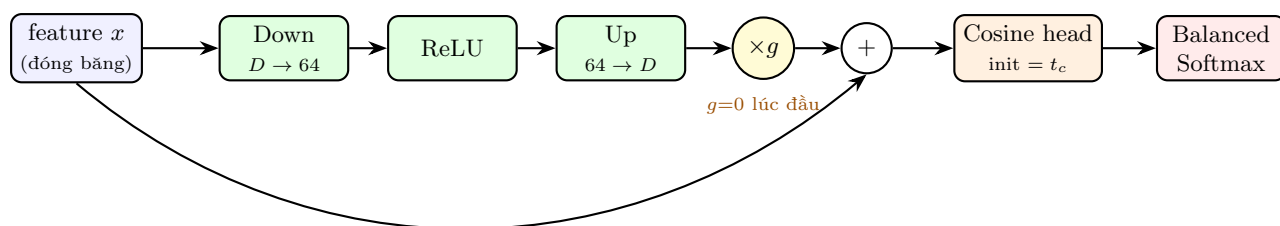
Tip-Adapter-F. Đây là biến thể có huấn luyện nhẹ: ta coi các khoá cache như trọng số của một lớp tuyến tính (khởi tạo bằng chính các khoá) rồi *fine-tune* vài epoch với Balanced Softmax (AdamW, $\text{lr} = 10^{-3}$, 20 epoch). Vì khởi tạo từ khoá, tại bước 0 nó *đúng bằng* Tip-Adapter không huấn luyện.

3.4.3 LIFT — thích nghi nhẹ với khởi tạo ngữ nghĩa (nguồn: vision-language)

Động lực. Thông điệp gốc của LIFT (Shi và cộng sự, ICML 2024) là *fine-tune nặng làm hại đuôi*. Nhóm giữ trọn tinh thần đó qua ba thành phần, minh hoạ ở Hình 3.2:

1. **Residual Adapter có cổng (gate).** Adapter có dạng $h = x + g \cdot \text{Up}(\text{ReLU}(\text{Down}(x)))$ với bottleneck 64 chiều và cổng g khởi tạo bằng 0. Tại bước 0, adapter là ánh xạ đồng nhất, nên mô hình *đúng bằng CLIP zero-shot* và chỉ rời khỏi đó khi việc huấn luyện thực sự cải thiện được đuôi. Adapter chỉ chiếm dưới 1% tham số so với backbone.
2. **Cosine classifier khởi tạo từ feature văn bản.** Logit bằng nhiệt độ nhân với cosine giữa đặc trưng và trọng số lớp, cả hai cùng chuẩn hoá L_2 . Trọng số được khởi tạo bằng *prototype văn bản* của tên lớp, nhờ đó ngay cả lớp 5 ảnh cũng bắt đầu từ một prototype có nghĩa thay vì ngẫu nhiên. Việc dùng cosine còn khiến logit *bất biến* với chuẩn đặc trưng, gỡ thêm một nguồn thiên lệch.
3. **Loss logit-adjusted.** Mô hình được huấn luyện bằng chính **BalancedSoftmaxLoss** (công thức 3.1) để đuôi không bị head lấn át, với AdamW, $\text{lr} = 10^{-3}$, weight decay 10^{-2} , 50 epoch, và chọn epoch tốt nhất theo balanced val.

Cần nói thêm rằng đây là biến thể LIFT ở *mức đặc trưng* (adapter chạy trên feature đã cache) nhằm giữ độ nhẹ; tuy vậy thông điệp “đóng băng backbone + học <1% tham số + loss nhận-biết-đuôi” vẫn được bảo toàn.



Hình 3.2: Kiến trúc LIFT: residual adapter có cổng ($g=0$ khởi tạo $\rightarrow$ bắt đầu từ zero-shot) + cosine head khởi tạo từ feature văn bản, huấn luyện bằng Balanced Softmax. Backbone đóng băng.

3.4.4 DINOv2-LIFT — mô hình nền tảng thị giác thứ hai (nguồn: thị giác)

Động lực. CLIP là một mô hình vision-language; câu hỏi đặt ra là nếu mượn một mô hình nền tảng *thị giác thuần* được huấn luyện *tự giám sát* (không ngôn ngữ) thì liệu nó có bắt được những chi tiết mà CLIP bỏ lỡ? Để trả lời, nhóm dùng **DINOv2** (dinov2_vits14, ViT-S/14, đặc trưng CLS 384 chiều). Vì DINOv2 không có bộ mã hoá văn bản nên cũng không có đường zero-shot; do đó đầu cosine của LIFT được khởi tạo bằng **nearest-class-mean** (trung bình đặc trưng train mỗi lớp) thay cho prototype văn bản. Phần còn lại — adapter và Balanced Softmax — giữ nguyên như LIFT, biến đây thành một thí nghiệm so sánh trực tiếp: cùng pipeline LIFT, chỉ đổi backbone.

3.4.5 Sinh đặc trưng bằng diffusion và feature mixup (nguồn: sinh dữ liệu / augment)

Feature diffusion (LDMLR-style). Thay vì sinh *ảnh*, nhóm huấn luyện một DDPM nhỏ *trên không gian đặc trưng*: một MLP khởi tạo có điều kiện theo lớp và bước thời gian (lịch β tuyến tính, 100 bước), học dự đoán nhiễu theo MSE. Khi sinh, ta lấy mẫu ngược từ Gauss để *tổng hợp đặc trưng cho lớp tail* đến một mức mục tiêu (mặc định là trung vị số mẫu), rồi huấn luyện LIFT trên hỗn hợp đặc trưng *thật* + *tổng hợp*. Ý tưởng là cân bằng lại tập huấn luyện ở mức feature, vốn rẻ hơn nhiều so với sinh ảnh.

Feature mixup (CMO ở mức feature). Ở đây nhóm chuyển ý tưởng CMO vào không gian đặc trưng: trộn lỗi một batch với các đối tác lấy từ luồng *thiên-đuôi* (trọng số $\propto 1/n_c$), $\tilde{x} = \lambda x_a + (1 - \lambda)x_b$, $\lambda \sim \text{Beta}(\alpha, \alpha)$, với mất mát hai-nhân-mềm; cơ chế này được kích hoạt qua `train_lift(..., mixup_alpha>0)`. Việc so sánh `lift_clip_mixup` (trộn feature thật) với `lift_clip_diff` (sinh feature) chính là để trả lời: kiểu augment nào giúp đuôi nhiều hơn?

3.4.6 Generalized Logit Adjustment (GLA) — đo công bằng

Động lực. Bản thân mô hình nền tảng cũng được huấn luyện trên dữ liệu web *lệch*, nên mang sẵn một thiên lệch nhân từ pretraining. GLA (Zhu và cộng sự, NeurIPS 2023) gỡ thiên lệch đó, qua đó tổng quát hoá Balanced Softmax sang “prior của foundation model”.

Công thức. Ta ước lượng log-bias từ *trung bình dự đoán* của mô hình trên tập đánh giá (transductive, không dùng nhãn), $b_c = \log(\frac{1}{N} \sum_i \text{softmax}(z(x_i))_c)$, rồi điều chỉnh $\hat{z} = z - \gamma b$ với cường độ γ chọn trên validation. Do lưới γ luôn chứa giá trị 0, GLA không bao giờ làm xấu đi kết quả được chọn.

3.4.7 Fusion nhận-biết-đuôi (kiểm bù trừ)

Động lực. Nếu mỗi nguồn tri thức mạnh ở một vùng khác nhau của đuôi thì việc gộp chúng đúng cách phải tốt hơn từng nguồn riêng lẻ — và đây cũng chính là phép kiểm *bù trừ*. Fusion ở đây tổng quát hoá tier-fusion/CLIP-fusion sang N chuyên gia, với **trọng số riêng cho từng nhóm-shot** (many/medium/few) tính chính trên validation. Vì validation thiếu lớp tail, trọng số nhóm *few* được *buộc theo nhóm medium* (ghi rõ trong cài đặt, không suy đoán).

Hai biến thể. Biến thể thứ nhất, `fusion_tailaware`, tìm trọng số trộn lỗi theo lưới đơn hình (simplex grid). Biến thể thứ hai, `fusion_greedy`, dùng *greedy ensemble selection* (Caruana và cộng sự, ICML 2004; theo tinh thần greedy soup của Wortsman và cộng sự, ICML 2022): bắt đầu từ chuyên gia đơn tốt nhất mỗi nhóm và chỉ *thêm* chuyên gia nếu việc đó cải thiện balanced accuracy của nhóm. Theo cách xây dựng, greedy luôn $\geq$ *chuyên gia đơn tốt nhất* trên validation, nên không bị tụt ở many/medium như lưới có thể gặp khi một chuyên gia (DINOv2) áp đảo.

3.5 Pipeline đề xuất của nhóm

Tổng hợp lại, “phương pháp đề xuất” của nhóm không phải một thuật toán đơn lẻ mà là một *khung nghiên cứu có thiết kế*: một pipeline thống nhất trải qua bốn giai đoạn, trong đó mỗi thành phần cũ được *tái dùng* làm công cụ cho giai đoạn sau (Bảng 3.2).

Bảng 3.2: Tái dùng có hệ thống các thành phần xuyên suốt bốn giai đoạn.

Thành phần (xuất xứ)	Được tái dùng làm gì ở Track 3
Balanced Softmax (Track 1, loss)	Loss huấn luyện của <i>mọi</i> expert LIFT / Tip-Adapter-F; được GLA tổng quát hoá sang prior của foundation model.
CMO (Track 1, dữ liệu)	Chuyển vào không gian feature thành <code>lift_clip_mixup</code> .
Tier-fusion / CLIP-fusion (Track 2)	Tổng quát hoá thành fusion N -expert nhận-biết-đuôi (kiểm bù trừ).
Mô hình cmo from-scratch (Track 1)	Mốc <i>đối chứng</i> “không tri thức ngoài” cho Phase 3.

Một số thiết kế then chốt cùng lý do của chúng có thể nêu ra như sau. Trước hết là việc *đóng băng backbone và chỉ học dưới 1% tham số*: với 5 ảnh/lớp, fine-tune nặng sẽ overfit và “quên” tri thức zero-shot ở đuôi — điều sẽ được kiểm chứng bằng Ablation B — trong khi việc cache feature một lần khiến toàn bộ Track 3 chạy được trong vài phút trên GPU Kaggle. Tiếp theo là *khởi tạo có ngữ nghĩa*: cosine head được khởi tạo từ feature văn bản (CLIP) hoặc class-mean (DINOv2), cho lớp 5 ảnh một điểm xuất phát hợp lý thay vì ngẫu nhiên. Bên cạnh đó, *loss nhận-biết-đuôi* (Balanced Softmax / GLA) được dùng

xuyên suốt nhằm bảo đảm gradient không bị head chi phối ở bất kỳ expert nào. Cuối cùng, *mọi siêu tham số đều được chọn trên validation* chứ không đụng tới test, để kết quả thu được là đáng tin cậy.

Chương 4

Các chiến lược đánh giá và độ đo áp dụng

Trên dữ liệu long-tailed, chọn sai thước đo gần như đồng nghĩa với sai từ gốc. Chương này trình bày giao thức đánh giá nghiêm ngặt mà nhóm áp dụng đồng nhất cho mọi phương pháp, ý nghĩa của từng độ đo, cùng cấu hình thực nghiệm và các mốc đối chứng (baseline).

4.1 Chiến lược chọn mô hình và giao thức đánh giá

4.1.1 Tách validation, không bao giờ dùng test

Như đã nêu ở Chương 2, một tỉ lệ `VAL_FRACTION = 0.1` được tách phân tầng từ train để *chọn checkpoint và tinh chỉnh siêu tham số*, còn tập test *chỉ* dùng cho con số báo cáo cuối cùng. Đây là điểm khác biệt so với một số báo cáo long-tail vô tình chọn mô hình ngay trên test, qua đó tạo ra kết quả lạc quan giả. Vì tập validation vẫn giữ hình dạng long-tail, việc chọn theo balanced accuracy là cần thiết để không thiên về head.

4.1.2 Hệ quả: validation thiếu lớp đuôi

Do mỗi lớp phải giữ ít nhất một ảnh train và lớp tail chỉ có 5 ảnh (giữ nguyên cả 5), tập validation thường *không chứa lớp tail*. Đây là một hạn chế cố hữu của giao thức và ảnh hưởng trực tiếp tới cách tinh chỉnh. Cụ thể, với các trọng số fusion theo nhóm-shot, nhóm *few* không có tín hiệu trên validation nên được **buộc theo trọng số của nhóm medium** (đây là quyết định được ghi rõ trong mã, không phải suy đoán). Tương tự, cường độ GLA và τ của τ -norm đều được chọn trên validation, với lưới luôn chứa giá trị “không can thiệp” (0) để trong trường hợp xấu nhất cũng không làm xấu kết quả.

4.2 Các độ đo đánh giá

Gọi $\text{recall}_c = \text{TP}_c / (\text{TP}_c + \text{FN}_c)$ là recall (độ nhạy) của lớp c . Các độ đo dưới đây được tính trong `src/evaluation/metrics.py`.

Thước đo *chính* là **balanced accuracy**, tức trung bình recall trên các lớp, $\text{bal-acc} = \frac{1}{C} \sum_c \text{recall}_c$. Mỗi lớp đóng góp như nhau bất kể tần suất, nên cải thiện ở tail không bị “nuốt” bởi head; và vì test cân bằng, đại lượng này trùng với accuracy thô. Đi kèm với nó là **few-shot accuracy** — trung bình recall *chỉ trên nhóm few-shot* — cùng các đại lượng tương tự cho many-shot và medium-shot, cho phép “mổ xẻ” hiệu năng theo mức độ hiếm. Ngoài ra, **macro-F1** (trung bình F1 trên các lớp) nhạy với cả precision lẫn recall của lớp hiếm.

Một độ đo đặc biệt hữu ích là **G-mean**, trung bình *nhân* của recall các lớp, $G = (\prod_c \text{recall}_c)^{1/C} = \exp(\frac{1}{C} \sum_c \log \text{recall}_c)$. Vì là trung bình nhân, chỉ cần *một* lớp có recall ≈ 0 cũng đủ kéo G-mean về gần 0, nên đây là thước đo khắc nghiệt nhất với hiện tượng bỏ rơi lớp tail. Trong kết quả, các phương pháp làm tail “chết” (chẳng hạn baseline) có G-mean rất nhỏ (cỡ 0.1), trong khi các phương pháp cứu được tail lại có G-mean cao hơn hẳn — một tín hiệu chẩn đoán rất trực quan. Sau cùng, **accuracy thô** và **MCC** cũng được tính để tham khảo, nhưng *không* dùng làm thước đo chính vì accuracy thô bị head thống trị.

Về mặt kỹ thuật, cảnh báo của sklearn “y_pred contains classes not in y_true” là vô hại trên validation long-tail (val thiếu một số lớp) và đã được chủ động chặn.

4.3 Cấu hình thực nghiệm

4.3.1 Môi trường

Toàn bộ huấn luyện chạy trên GPU Kaggle (NVIDIA T4, có cấu hình 2×T4); cờ USE_MULTI_GPU chia batch trích đặc trưng (chỉ forward, an toàn) lên hai GPU. Framework là PyTorch (2.6); cần lưu ý PyTorch 2.6 đặt torch.load mặc định weights_only=True, nên toàn bộ điểm nạp checkpoint trong project dùng weights_only=False và lưu scalar dưới dạng float Python. Các thư viện ngoài gồm open_clip_torch (CLIP), torch.hub (DINOv2), transformers (LLM Qwen2.5-3B-Instruct) và scikit-learn (độ đo, t-SNE); riêng CLIP/DINOv2/LLM cần Internet trên Kaggle để tải trọng số lần đầu. Để bảo đảm tái lập, nhóm cố định hạt giống (42) và đặt cudnn.deterministic=True, trong khi episode của meta-learning dùng một bộ sinh ngẫu nhiên riêng.

4.3.2 Siêu tham số huấn luyện

Bảng 4.1 tổng hợp các siêu tham số chính, lấy trực tiếp từ mã nguồn.

Bảng 4.1: Siêu tham số chính của các phương pháp (theo mã nguồn).

Phương pháp / thành phần	Cấu hình tối ưu hoá	Ghi chú
Track 1 (chung)	SGD, lr 0.1, momentum 0.9, wd 5×10^{-4}	200 epoch, cosine, batch 128
SupCon pretrain	SGD, lr 0.5, wd 10^{-4} , $\tau=0.07$	200 epoch; head 512→512→128
cRT (decoupling/supcon)	lr 0.1, sampler cân bằng	10 epoch, đóng băng BN
Meta (ProtoNet)	Adam, lr 10^{-3}	5-way 5-shot 15-query, 100 episode/epoch
CMO/Mixup/CutMix	$\alpha=1.0$, xác suất trộn 0.5	tiêu chí Balanced Softmax
Tip-Adapter	grid α, β trên val	training-free; cache cân bằng
Tip-Adapter-F	AdamW, lr 10^{-3}	20 epoch, Balanced Softmax
LIFT / DINOv2-LIFT	AdamW, lr 10^{-3} , wd 10^{-2}	50 epoch, bottleneck 64, cosine
Feature diffusion	AdamW, lr 10^{-3}	100 bước, lịch $\beta \in [10^{-4}, 0.02]$
CLIP fine-tune (ablation)	AdamW, lr 10^{-4}	10 epoch; depth: linear/last-block/full
Backbone nền tảng	CLIP ViT-B/32 (512d); DINOv2 ViT-S/14 (384d)	đóng băng

Trước mỗi lần chạy đầy đủ, nhóm đều thực hiện một bước smoke-test nhanh (MAX_TRAIN_SAMPLES=2000, EPOCHS=5) để bảo đảm pipeline thông suốt, rồi mới chạy đầy đủ với EPOCHS=200.

4.4 Các mốc đối chứng (baselines)

Để định vị chính xác đóng góp, báo cáo sử dụng nhiều lớp baseline khác nhau. *Baseline tuyệt đối* là ResNet-18 + cross-entropy, đóng vai mốc “không can thiệp” — mọi cải thiện so với nó đều cho thấy giá trị của can thiệp tail-aware. *Trần from-scratch* là cmo, phương pháp huấn luyện-từ-đầu mạnh nhất, đồng thời là “đối chứng không tri thức ngoài” cho toàn bộ Track 3. *Mốc zero-shot* là CLIP zero-shot, đóng vai mốc sàn của track tri thức ngoài (không huấn luyện gì). Cuối cùng là các *mốc ablation*: chuỗi DINOv2 (linear-probe+CE → linear-probe+Balanced-Softmax → LIFT đầy đủ) dùng để tách đóng góp của *loss* và của *kiến trúc*, còn chuỗi độ-sâu fine-tune CLIP dùng để kiểm luận điểm “fine-tune nặng hại đuôi”. Việc tách thành hai bảng riêng — *leaderboard from-scratch* và *bảng tri thức ngoài* — là có chủ đích: các phương pháp dùng tri thức ngoài vốn không cùng setup với from-scratch, nên trình bày riêng sẽ “chính danh” hơn khi so sánh.

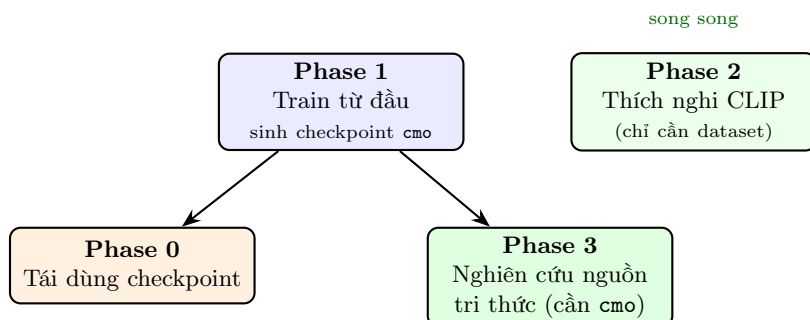
Chương 5

Huấn luyện và đánh giá

Chương này trình bày quy trình huấn luyện thực tế, kết quả định lượng trên cả hai bộ dữ liệu, các nghiên cứu loại trừ (ablation), phân tích định tính và thảo luận. Mọi con số đều được lấy trực tiếp từ các tệp kết quả trong `outputs/cifar/*.csv` và `outputs/cub_200_2011/*.csv`.

5.1 Quy trình huấn luyện theo bốn giai đoạn

Bốn giai đoạn được chạy liên hoàn trong `run_pipeline.ipynb` (mỗi phase là một hàm có phạm vi riêng, dùng chung cấu hình ở đầu file), theo thứ tự phụ thuộc minh hoạ ở Hình 5.1.



Hình 5.1: Phụ thuộc giữa các giai đoạn. Phase 1 sinh checkpoint `cmo` làm đối chứng cho Phase 0 và Phase 3; Phase 2 độc lập (chỉ cần dataset) nên chạy song song được.

Mỗi lần chạy ghi lại `config.json`, lịch sử theo epoch (`metrics.csv`), checkpoint tốt nhất (`best_model.pt`) cùng các hình (confusion matrix, t-SNE) vào `outputs/<method>/`; cuối cùng, `compare_runs` dựng bảng so sánh được sắp theo balanced accuracy.

5.2 Kết quả định lượng trên CIFAR-100-LT

5.2.1 Track 1 — Leaderboard huấn luyện từ đầu

Bảng 5.1 cho thấy một bức tranh khá rõ theo tầng can thiệp. Baseline đạt 0.401 balanced accuracy nhưng few-shot chỉ vón vện 0.079 — đuôi gần như chết. Can thiệp ở tầng *mất mát* (balanced_softmax) nâng few-shot lên 0.181 (khoảng 2.3 lần), còn can thiệp đồng thời ở *dữ liệu* và *mất mát* (cmo) đạt cao nhất với balanced accuracy 0.468 và few-shot 0.299 (gần 4 lần baseline). Một điểm thú vị là cmo và balanced_softmax chấp nhận *hy sinh một chút head* (many giảm từ 0.707 xuống 0.641) để *cứu lấy đuôi* — đúng với bản chất đánh đổi của bài toán long-tail.

Bảng 5.1: CIFAR-100-LT — Track 1, huấn luyện từ đầu (`comparison_train_from_scratch.csv`). Sắp theo balanced accuracy.

Phương pháp	bal-acc	many	medium	few
cmo	0.468	0.641	0.441	0.299
balanced_softmax	0.441	0.681	0.424	0.181
decoupling	0.407	0.706	0.375	0.094
baseline	0.401	0.707	0.371	0.079
supcon	0.367	0.668	0.361	0.021
meta	0.351	0.558	0.390	0.064

Như vậy trần của việc học-từ-đầu rơi vào khoảng 0.47 balanced accuracy (tail $\approx$ 0.30). SupCon kém kỳ vọng ở đuôi (few 0.021) bởi với 5 ảnh/lớp, số cặp dương quá ít để hình thành cụm tương phản tốt cho tail; còn Meta (ProtoNet) yếu ở 100-way như đã dự đoán, vì vốn được thiết kế cho few-way.

5.2.2 Track 2 — Tái dùng checkpoint

Bảng 5.2 trình bày hiệu quả của các kỹ thuật reuse. CLIP fusion (`vlm_fusion`) dẫn đầu (0.681) và vượt cả hai chuyên gia thành phần, cho thấy giá trị của tri thức ngôn ngữ ngay cả khi mới chỉ “trộn”. Các kỹ thuật thuần-thị-giác (ensemble, tier_fusion, τ -norm) chỉ nâng nhẹ so với mô hình đơn và chủ yếu ở head; trong nhóm này, τ -norm cải thiện đuôi rõ nhất (0.364).

Bảng 5.2: CIFAR-100-LT — Track 2, tái dùng checkpoint (`comparison_phase0.csv`).

Kỹ thuật	bal-acc	many	medium	few
vlm_fusion (CLIP + vision)	0.681	0.810	0.645	0.571
clip_only (zero-shot)	0.627	0.638	0.605	0.638
tau_norm	0.543	0.720	0.521	0.364
ensemble_tta	0.494	0.772	0.493	0.172
tier_fusion	0.489	0.775	0.513	0.126

Có một điểm về tính nhất quán cần nói rõ ở đây. Bộ checkpoint dùng cho Phase 0 cho số mô hình-đơn *cao hơn* leaderboard from-scratch ở Bảng 5.1 (ví dụ `cmo` đạt 0.514 ở Phase 0 so với 0.468 ở Track 1), nhiều khả năng do bộ này dùng `pretrained=True`. Vì vậy Bảng 5.2 nên được đọc như phần *minh họa các kỹ thuật reuse* chứ không trộn trực tiếp con số với Track 1 hay Track 3; đối chứng from-scratch chuẩn vẫn là Track 1 và Track 3.

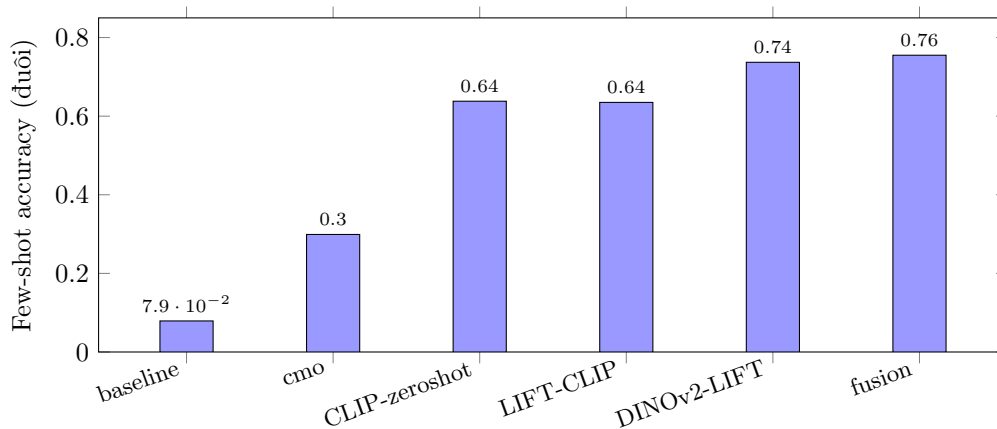
5.2.3 Track 3 — Thách nghi mô hình nền tảng (kết quả chính)

Đây là bảng quan trọng nhất của báo cáo (Bảng 5.3). Track tri thức ngoài tạo ra một bước nhảy quyết định: từ trần from-scratch 0.47 lên **0.82** balanced accuracy, với few-shot tăng từ 0.30 lên **0.755**. Mô hình nền tảng thị giác thứ hai (`dino_lift`, 0.811) vượt rõ LIFT-CLIP (0.717), còn fusion gộp các expert đạt cao nhất (0.823) mà *không tụt ở bất kỳ nhóm-shot nào*.

Bảng 5.3: CIFAR-100-LT — Track 3, thích nghi mô hình nền tảng (`comparison_phase3.csv` & `comparison_vlm_phase2.csv`). Sắp theo balanced accuracy.

Expert	Nguồn tri thức	bal-acc	many	medium	few
fusion_greedy	gộp	0.823	0.890	0.813	0.755
fusion_tailaware	gộp	0.820	0.893	0.803	0.753
dino_lift	thị giác thứ hai	0.811	0.872	0.813	0.737
lift (CLIP)	vision-language	0.718	0.782	0.727	0.635
tip_adapter	truy hồi	0.668	0.679	0.648	0.678
tip_adapter_f	truy hồi (FT)	0.643	0.821	0.672	0.400
clip_llm	ngôn ngữ	0.626	0.618	0.606	0.658
clip_zeroshot	—	0.627	0.638	0.605	0.638
cmo (đối chứng)	nội tại	0.462	0.614	0.443	0.307

Hình 5.2 trực quan hoá thông điệp cốt lõi: few-shot accuracy *tăng đơn điệu* qua bốn mốc đại diện cho mạch tiếp cận.



Hình 5.2: CIFAR-100-LT: few-shot accuracy tăng đơn điệu từ baseline (0.079) → cmo (trần from-scratch, 0.299) → track tri thức ngoài (CLIP 0.638, LIFT, DINOv2 0.737, fusion 0.755). Đuôi được cứu chủ yếu nhờ *mượn tri thức ngoài*.

5.3 Kết quả định lượng trên CUB-200-LT (fine-grained)

Trên miền fine-grained khó hơn, khoảng cách giữa hai track còn nới rộng thêm (Bảng 5.4). Huấn luyện-từ-đầu gần như vô dụng, với balanced accuracy chỉ quanh mức 0.20–0.27 (cmo 0.195, baseline 0.258), trong khi track nền tảng đạt **0.85** — một bằng chứng mạnh cho thấy *tri thức ngoài là thiết yếu* khi dữ liệu vừa ít vừa khó. Một lần nữa **dino_lift** (0.841) bỏ xa LIFT-CLIP (0.670), thậm chí còn mạnh hơn so với trên CIFAR; điều này phù hợp với việc DINOv2 (patch nhỏ 14, tự giám sát) bắt chỉ tiết fine-grained rất tốt.

Bảng 5.4: CUB-200-LT — Track 3, thích nghi mô hình nền tảng (`comparison.csv`).

Expert	Nguồn tri thức	bal-acc	many	medium	few
fusion_greedy	gộp	0.849	0.858	0.824	0.864
fusion_tailaware	gộp	0.846	0.859	0.824	0.854
dino_lift	thị giác thứ hai	0.841	0.851	0.812	0.859
lift (CLIP)	vision-language	0.670	0.741	0.650	0.593
tip_adapter_f	truy hồi (FT)	0.649	0.758	0.623	0.529
tip_adapter	truy hồi	0.565	0.637	0.521	0.516
clip_llm	ngôn ngữ	0.484	0.518	0.458	0.466
clip_zeroshot	—	0.460	0.494	0.433	0.445
cmo (đối chứng)	nội tại	0.195	0.256	0.186	0.120

5.4 Nghiên cứu loại trừ (Ablation Study)

5.4.1 Ablation A — đóng góp của pipeline so với backbone (DINOv2)

Câu hỏi đặt ra là: trong kết quả cuối cùng, bao nhiêu phần đến từ *backbone mạnh* (DINOv2) và bao nhiêu đến từ *pipeline của nhóm* (loss nhận-biết-đuôi + adapter/cosine head)? Để tách bạch, nhóm dựng một chuỗi ba mốc trên cùng đặc trưng DINOv2 (Bảng 5.5). Trên CIFAR, chỉ riêng việc đổi cross-entropy sang Balanced Softmax đã kéo few-shot từ 0.020 lên 0.325 (đóng góp của *loss*); thêm adapter và cosine init đẩy tiếp lên 0.737 (đóng góp của *kiến trúc*). Trên CUB hiệu ứng còn rõ rệt hơn (0.077 → 0.675 → 0.859). Kết luận rút ra là *backbone mạnh thôi chưa đủ*: nếu chỉ gắn một đầu tuyến tính ngây thơ thì đuôi vẫn sụp, và phần lớn giá trị thực sự đến từ pipeline tail-aware của nhóm.

Bảng 5.5: Ablation A trên đặc trưng DINOv2: tách đóng góp của loss và kiến trúc.

Mốc	CIFAR-100-LT		CUB-200-LT	
	bal-acc	few	bal-acc	few
dino_linear_probe (Linear + CE)	0.463	0.020	0.490	0.077
dino_linear_probe_bs (Linear + Balanced-Softmax)	0.657	0.325	0.744	0.675
dino_lift (+adapter +cosine init)	0.811	0.737	0.841	0.859

5.4.2 Ablation B — độ sâu fine-tune CLIP: fine-tune nặng làm hại đuôi

Đây là ablation nhằm biện minh cho thiết kế *đóng băng*. Nhóm fine-tune CLIP ở ba độ sâu tăng dần — chỉ head, rồi block cuối, rồi toàn bộ visual encoder — với Balanced Softmax (Bảng 5.6, trên CIFAR). Balanced accuracy có tăng theo độ sâu (0.461 → 0.536 → 0.558), nhưng few-shot lại tệ đi và luôn ở mức rất thấp (cao nhất chỉ 0.128). So với LIFT *đóng băng* (few-shot 0.635), mọi cấu hình fine-tune đều thua xa ở đuôi. Kết quả này xác nhận đúng luận điểm của LIFT: với 5 ảnh/lớp, việc lan truyền gradient vào backbone gây overfit và xói mòn tri thức zero-shot ở các lớp hiếm.

Bảng 5.6: Ablation B trên CIFAR-100-LT: độ sâu fine-tune CLIP (Balanced Softmax). G-mean rất nhỏ cho thấy nhiều lớp tail bị “chết”.

Cấu hình	bal-acc	many	few	G-mean
ft_linear_probe	0.461	0.785	0.086	0.004
ft_last_block	0.536	0.850	0.128	0.005
ft_full_ft	0.558	0.874	0.111	0.007
lift (đóng băng)	0.718	0.782	0.635	0.697

5.4.3 Augment đặc trưng: sinh dữ liệu và mixup không giúp

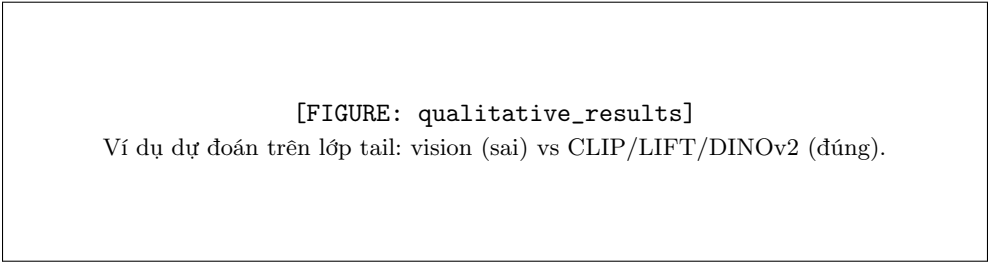
Khi so sánh ba biến thể trên cùng nền LIFT-CLIP (Bảng 5.7), cả *sinh đặc trưng bằng diffusion* (`lift_clip_diff`) lẫn *feature mixup* (`lift_clip_mixup`) đều *thấp hơn* LIFT-CLIP gốc trên *cả hai* dataset. Đây là một kết quả âm trung thực: khi đặc trưng nền tảng đã đủ mạnh, việc augment thêm chủ yếu đưa vào nhiễu thay vì thông tin mới. Riêng diffusion còn làm tail tệ đi rõ rệt trên CIFAR (few 0.505).

Bảng 5.7: Augment đặc trưng so với LIFT-CLIP gốc (balanced accuracy).

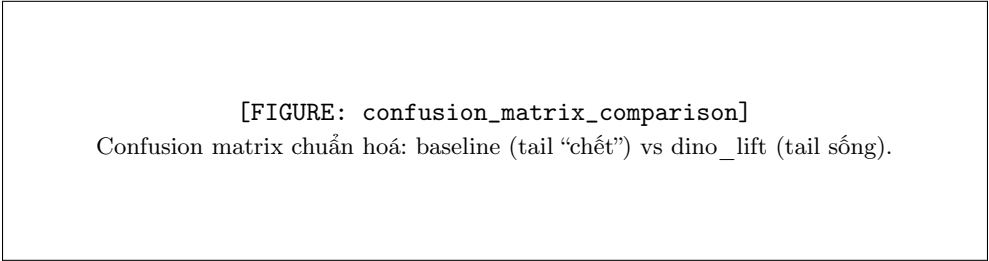
Phương pháp	CIFAR-100-LT	CUB-200-LT
lift_clip (gốc)	0.717	0.670
lift_clip_mixup (feature mixup)	0.698	0.654
lift_clip_diff (diffusion)	0.675	0.655

5.5 Kết quả định tính

Một “slide demo đuôi” điển hình minh hoạ rất rõ giá trị của tri thức ngoài: với một ảnh thuộc lớp hiếm, mô hình thị giác from-scratch thường dự đoán sai (gán nhầm sang một lớp head trông tương tự), trong khi LIFT/DINOv2 — nhờ khởi tạo ngữ nghĩa và loss nhận-biết-đuôi — lại dự đoán đúng. Các trường hợp như vậy cho thấy bằng trực giác vì sao tri thức ngoài cứu được đuôi. Những hình tương ứng (confusion matrix, t-SNE của đặc trưng) được sinh ra từ pipeline đánh giá và có thể chèn vào các vị trí dưới đây.



Hình 5.3: Phân tích định tính trên các lớp đuôi (placeholder — chèn ảnh từ pipeline đánh giá).



Hình 5.4: So sánh confusion matrix: phương pháp từ đầu bỏ rơi đuôi, track nền tảng phân bố đều hơn.

5.6 Thảo luận

5.6.1 Các phát hiện chính (nhất quán trên cả hai dataset)

- 1. **Tri thức ngoài là bước nhảy quyết định.** Trên CIFAR, kết quả đi từ from-scratch 0.47 lên nền tảng 0.82; trên CUB là từ 0.20–0.27 lên 0.85. Chênh lệch càng lớn ở miền khó (fine-grained CUB), nơi học-từ-đầu gần như thất bại.
- 2. **Nguồn cứu đuôi nhất là mô hình nền tảng thị giác thứ hai (DINOv2).** `dino_lift` vượt LIFT-CLIP ở cả hai dataset (0.811 vs 0.718; 0.841 vs 0.670) và là trụ cột của fusion. DINOv2 được

huấn luyện tự giám sát với patch 14, nên bắt chi tiết fine-grained tốt hơn CLIP ViT-B/32 ở ảnh nhỏ.

3. **Ngôn ngữ (LLM) giúp ít.** `clip_llm` gần như ngang `clip_zeroshot` (CIFAR 0.626 vs 0.627; CUB 0.484 vs 0.460, chỉ nhích nhẹ). Mô tả giàu không bù được hạn chế của CLIP ViT-B/32 ở ảnh nhỏ và dữ liệu fine-grained.
4. **Sinh dữ liệu và mixup không giúp** (cả hai dataset đều thấp hơn LIFT-CLIP gốc): khi đặc trưng đã mạnh, augment thêm chỉ đưa vào nhiễu.
5. **Đóng góp của pipeline đo được rõ ràng** (Ablation A): chỉ riêng Balanced Softmax đã cứu đuôi từ ~ 0.02 lên 0.33 (CIFAR) và từ 0.08 lên 0.68 (CUB); thêm adapter cùng cosine init đẩy tiếp lên 0.74/0.86.
6. **Fine-tune nặng làm hại đuôi** (Ablation B): full-FT cho few-shot 0.111 so với LIFT đồng bằng 0.635 — một biện minh trực tiếp cho thiết kế đóng băng.
7. **Bù trừ có nhưng ở mức nhẹ và an toàn.** `fusion_greedy` vượt expert đơn tốt nhất ở cả hai dataset ($0.823 > 0.811$; $0.849 > 0.841$) mà không tụt ở nhóm nào, vì greedy bảo đảm $\geq$ best-single trên validation cho từng nhóm.

5.6.2 Điểm mạnh và hạn chế

Về điểm mạnh, nghiên cứu có một khung thống nhất, đa-dataset, với giao thức chọn-mô-hình-trên-val nghiêm ngặt; các kết luận nhất quán trên hai miền và hai quy mô lớp; có ablation tách bạch đóng góp và trung thực với cả các kết quả âm.

Để bảo đảm minh bạch, nhóm cũng nêu rõ hai ghi chú thực nghiệm (caveat) liên quan đến *breakdown*, vốn không ảnh hưởng tới balanced accuracy hay các kết luận. Thứ nhất, *CIFAR Phase 0* chạy trên một bộ checkpoint cho số mô hình-đơn cao hơn leaderboard from-scratch (có thể do `pretrained=True`), nên Bảng 5.2 chỉ nên xem là minh họa kỹ thuật reuse, không so trực tiếp con số với Track 1/3. Thứ hai, *CUB from-scratch* được chấm khi ngưỡng nhóm-shot còn ở mặc định (100/20), nên nhóm *many* rộng và cột many/medium/few của riêng nhóm này *không hợp lệ*; tuy vậy balanced accuracy không phụ thuộc cách chia nhóm nên vẫn đúng, và track nền tảng (Bảng 5.4) đã dùng đúng ngưỡng 20/10. Cả hai điểm này chỉ liên quan đến breakdown; balanced accuracy và mọi kết luận đều được giữ nguyên.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Báo cáo nghiên cứu bài toán nhận dạng ảnh trên dữ liệu đuôi-dài, nơi một số ít lớp phổ biến áp đảo rất nhiều lớp hiếm (đuôi chỉ 5 ảnh/lớp), theo một mạch tiếp cận có cấu trúc: *đo cho đúng* → *sửa bộ phân loại* → *mượn tri thức ngoài*. Trên hai bộ dữ liệu khác biệt cả về quy mô lẫn miền — CIFAR-100-LT (IF=100) và CUB-200-LT (fine-grained, IF=10) — nhóm rút ra một kết luận nhất quán: với đuôi ít ảnh, chìa khoá *không* nằm ở việc huấn luyện một mạng lớn hơn từ đầu, mà ở việc **mượn và thích nghi nhẹ một mô hình nền tảng đóng băng bằng loss nhận-biết-đuôi**. Trong khi track from-scratch chỉ chạm trần khoảng 0.47 balanced accuracy (CIFAR) và 0.20–0.27 (CUB), track nền tảng đạt tới **0.82 / 0.85**. Xét riêng các nguồn tri thức ngoài, **mô hình nền tảng thị giác thứ hai (DINOv2) mang lại lợi ích lớn nhất**; ngôn ngữ (LLM) giúp được ít, còn sinh dữ liệu và mixup gần như không giúp.

6.2 Những đóng góp chính của nhóm

- **Xây dựng** một khung thực nghiệm thống nhất, đa-dataset cho long-tailed recognition, với giao thức chọn-mô-hình-trên-validation nghiêm ngặt và bộ đo tail-aware (balanced accuracy, few-shot accuracy, G-mean).
- **Triển khai và đối chứng** đầy đủ ba lớp phương pháp (huấn luyện từ đầu, tái dùng checkpoint, thích nghi mô hình nền tảng đóng băng) trên cùng một thước đo.
- **Thực hiện** một nghiên cứu so sánh có hệ thống ba nguồn tri thức ngoài (ngôn ngữ / thị giác thứ hai / sinh dữ liệu) trên cùng một đuôi, kèm công cụ đo công bằng (GLA) và phép kiểm bù trừ (fusion nhận-biết-đuôi), có đối chiếu với mốc from-scratch.
- **Chứng minh bằng số** (qua hai ablation) rằng phần lớn giá trị đến từ *pipeline tail-aware* của nhóm chứ không chỉ từ backbone, và rằng *fine-tune nặng làm hại đuôi* — qua đó biện minh cho thiết kế đóng băng.
- **Báo cáo trung thực** cả các kết quả âm lẫn các ghi chú thực nghiệm, bảo đảm tính khách quan.

6.3 Hạn chế

Đề tài còn một số hạn chế cần nhìn nhận thẳng thắn. Trước hết là về backbone: do ràng buộc GPU Kaggle, nhóm chỉ dùng CLIP ViT-B/32 (cấu hình nhẹ nhất), nên track ngôn ngữ (CLIP-LLM) có thể bị đánh giá thấp — một CLIP lớn hơn (ViT-L/14) hoàn toàn có thể làm thay đổi cán cân giữa ngôn ngữ và thị giác. Tiếp đến là vấn đề validation thiếu lớp đuôi: vì lớp tail chỉ có 5 ảnh và được giữ nguyên cho train, việc tinh chỉnh trên validation không có tín hiệu trực tiếp cho nhóm few, buộc trọng số nhóm này phải lấy theo medium. Ngoài ra, IF của CUB bị giới hạn ở mức ≤ 50 do pool chỉ khoảng 50 ảnh/lớp, nên không gất bằng CIFAR và được trình bày như một miền fine-grained. Cuối cùng là hai caveat về breakdown đã nêu trong phần thảo luận (Chương 5) liên quan tới bộ checkpoint Phase 0 và ngưỡng nhóm-shot của CUB from-scratch; cả hai chỉ ảnh hưởng tới breakdown chứ không ảnh hưởng balanced accuracy.

6.4 Hướng phát triển

Từ những hạn chế trên, một số hướng mở rộng tự nhiên có thể kể tới. Hướng đầu tiên là thử các backbone nền tảng lớn hơn (CLIP ViT-L/14, DINOv2 ViT-L/14) để kiểm tra xem kết luận “DINOv2 > CLIP” có còn giữ nguyên khi tăng dung lượng mô hình hay không. Hướng thứ hai là chạy lại một đối chứng from-scratch *nhất quán hoàn toàn* — Phase 0 trên đúng checkpoint `pretrained=False`, và from-scratch CUB với ngưỡng nhóm-shot 20/10 — nhằm thu được một bảng “sạch” tuyệt đối. Bên cạnh đó, có thể nghiên cứu cách *ước lượng phân phối đuôi* (hoặc dựng một validation tổng hợp cho đuôi) để chọn siêu tham số fusion cho nhóm few mà không phải buộc theo medium. Sau cùng, nhóm muốn khảo sát các cơ chế bù trừ sâu hơn giữa các nguồn tri thức (ví dụ kết hợp ở mức feature thay vì mức xác suất) và mở rộng nghiên cứu sang các dataset long-tail quy mô lớn hơn như ImageNet-LT hay iNaturalist.

Phụ lục

A. Cấu trúc mã nguồn

```
src/  
  datasets/   cifar_lt.py (đọc dữ liệu, tách split, nhóm-shot), augment.py, episodic.py  
  models/    backbone.py (build_encoder, CIFAR stem), baseline.py, projection.py, prototype.py  
  trainers/   engine.py (vòng train/eval), losses.py (Balanced Softmax),  
              baseline_trainer.py, decoupling_trainer.py, contrastive_trainer.py,  
              augment_trainer.py, meta_trainer.py, classifier.py  
  experts/    clip_expert.py, tip_adapter.py, lift.py, clip_finetune.py,  
              llm_prompts.py, dino_expert.py, feature_diffusion.py,  
              feature_mixup.py, gla.py  
  evaluation/ metrics.py, ensemble.py, posthoc.py, fusion.py, visualize.py  
  notebooks/ run_pipeline.ipynb (4 phase, toggle), run_all_methods.ipynb,  
              phase0_reuse.ipynb, phase2_clip_adapt.ipynb, phase3_knowledge_sources.ipynb  
  data/       prepare_datasets.py (CIFAR-100-LT), prepare_cub_lt.py (CUB-200-LT)  
  outputs/    cifar/*.csv, cub_200_2011/*.csv (kết quả thô), figures/ (hình EDA)
```

B. Tái lập kết quả

Trình tự tối giản (một máy, tuần tự): `prepare_datasets.py` → `run_all_methods.ipynb` (sinh checkpoint, gồm cmo) → `phase0_reuse.ipynb` → `phase2_clip_adapt.ipynb` → `phase3_knowledge_sources.ipynb`.
Hoặc chạy tất cả trong `run_pipeline.ipynb` với các toggle `RUN_PHASE1/0/2/3`. Đổi sang CUB-200-LT chỉ cần đặt `DATA_DIR`, ngưỡng nhóm-shot `MANY_THRESHOLD=20`, `FEW_THRESHOLD=10`, và tắt `USE_CMO`.

C. Nguồn số liệu

Mọi bảng trong báo cáo lấy trực tiếp từ: `outputs/cifar/train_from_scratch/comparison_train_from_scratch.csv` (Track 1), `outputs/cifar/pretrained/comparison_phase0.csv` (Track 2), `outputs/cifar/comparison_phase3.csv` và `comparison_vlm_phase2.csv` (Track 3), `outputs/cub_200_2011/comparison.csv` và `knowledge_sources.csv` (CUB). Thống kê dữ liệu CIFAR từ `outputs/cifar/summary.csv`; hình EDA từ `outputs/figures/`.

Tài liệu tham khảo

- [1] J. Ren, C. Yu, S. Sheng, X. Ma, H. Zhao, S. Yi, and H. Li. *Balanced Meta-Softmax for Long-Tailed Visual Recognition*. NeurIPS, 2020.
- [2] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, and Y. Kalantidis. *Decoupling Representation and Classifier for Long-Tailed Recognition*. ICLR, 2020.
- [3] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan. *Supervised Contrastive Learning*. NeurIPS, 2020.
- [4] J. Snell, K. Swersky, and R. Zemel. *Prototypical Networks for Few-shot Learning*. NeurIPS, 2017.
- [5] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. *mixup: Beyond Empirical Risk Minimization*. ICLR, 2018.
- [6] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo. *CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features*. ICCV, 2019.
- [7] S. Park, Y. Hong, B. Heo, S. Yun, and J. Y. Choi. *The Majority Can Help the Minority: Context-rich Minority Oversampling for Long-tailed Classification*. CVPR, 2022.
- [8] A. Radford, J. W. Kim, C. Hallacy, et al. *Learning Transferable Visual Models From Natural Language Supervision (CLIP)*. ICML, 2021.
- [9] R. Zhang, W. Zhang, R. Fang, P. Gao, K. Li, J. Dai, Y. Qiao, and H. Li. *Tip-Adapter: Training-free Adaption of CLIP for Few-shot Classification*. ECCV, 2022.
- [10] J. Shi, et al. *Long-Tail Learning with Foundation Model: Heavy Fine-Tuning Hurts (LIFT)*. ICML, 2024.
- [11] S. Pratt, I. Covert, R. Liu, and A. Farhadi. *What does a platypus look like? Generating customized prompts for zero-shot image classification (CuPL)*. ICCV, 2023.
- [12] M. Oquab, T. Darcet, T. Moutakanni, et al. *DINOv2: Learning Robust Visual Features without Supervision*. TMLR, 2023.
- [13] B. Han, et al. *Latent-based Diffusion Model for Long-tailed Recognition (LDMLR)*. 2024.
- [14] B. Zhu, K. Tang, Q. Sun, and H. Zhang. *Generalized Logit Adjustment: Calibrating Fine-tuned Models by Removing Label Bias in Foundation Models*. NeurIPS, 2023.
- [15] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes. *Ensemble Selection from Libraries of Models*. ICML, 2004.
- [16] M. Wortsman, G. Ilharco, S. Y. Gadre, et al. *Model Soups: Averaging Weights of Multiple Fine-tuned Models Improves Accuracy without Increasing Inference Time*. ICML, 2022.
- [17] K. He, X. Zhang, S. Ren, and J. Sun. *Deep Residual Learning for Image Recognition*. CVPR, 2016.
- [18] J. Ho, A. Jain, and P. Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.
- [19] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie. *The Caltech-UCSD Birds-200-2011 Dataset*. Technical Report CNS-TR-2011-001, Caltech, 2011.
- [20] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images (CIFAR)*. Technical Report, University of Toronto, 2009.